

REQUESTS | Engineering Specification

01. Architectural Design

Project Definition and Architecture

The project is a modular, object-oriented library implementing a Hypertext Transfer Protocol (HTTP) client, organized according to the Repository Pattern and the Model-View-Controller (MVC) architectural pattern.

Repository Pattern

The project's structure suggests a separation of concerns between the core library functionality, residing in the `src/requests` directory, and the test suite, located in the `tests` directory. This separation is indicative of the Repository Pattern, which aims to abstract data access and encapsulate the data storage and retrieval logic.

Model-View-Controller (MVC) Pattern

Within the `src/requests` directory, the project's structure and file naming conventions suggest adherence to the MVC pattern. The main components of this pattern are:

- **Models:** Represented by the `models.py` file, which likely contains classes and data structures for handling HTTP requests and responses.
- **Controllers:** Embodied by the `sessions.py` file, which probably manages the interaction between the models and the external world, governing the creation, modification, and termination of HTTP sessions.
- **Utilities and Helpers:** The `utils.py` and `_internal_utils.py` files contain supplementary functions and classes that support the core functionality of the library, such as authentication mechanisms (`auth.py`) and other low-level utilities.

Test-Driven Development (TDD)

The project's test suite, residing in the `tests` directory, is structured to reflect the different aspects of the library's functionality, including:

- **Unit Tests:** Targeting specific components, such as `test_requests.py`, `test_utils.py`, and `test_lowlevel.py`.
- **Integration Tests:** Covering more comprehensive scenarios, like `test_testserver.py` and `test_hooks.py`.
- **Package Tests:** Ensuring the library's packaging and distribution are correctly handled (`test_packages.py`).

Documentation

The `docs` directory contains configuration files for generating documentation, such as `conf.py`, indicating that the project incorporates automated documentation generation, likely using tools like Sphinx.

In summary, the project's architecture is designed to promote modularity, maintainability, and scalability, with clear separation of concerns between the core library functionality, testing, and documentation.

02. System Workflow

The following diagram outlines the high-level call sequence and module dependencies.

```
sequenceDiagram
```

```
autonumber
```

```
Note over src/requests/__init__.py, src/requests/sessions.py
```

The primary ENTRY file is identified as `src/requests/\_\_init\_\_.py` because it is the typical entry point for a Python package. The double underscore `\_\_init\_\_.py` file is used to initialize the package and serves as the primary entry point for users importing the package.

The primary CORE logic file is identified as `src/requests/sessions.py` because sessions are the core concept in the requests library, responsible for managing connections, sending requests, and receiving responses. This file is likely to contain the primary logic for handling HTTP requests and sessions.

```
Critical Path
tests_test_requests_py->>_git-blame-ignore-revs: invokes
tests_test_requests_py->>_gitignore: invokes
tests_test_requests_py->>_pre-commit-config.yaml: invokes
tests_test_requests_py->>_readthedocs.yaml: invokes
tests_test_requests_py->>requirements-dev.txt: invokes
tests_test_requests_py->>docs_requirements.txt: invokes
tests_test_requests_py->>docs_community_out-there.rst: invokes
tests_test_requests_py->>docs_community_recommended.rst: invokes
tests_test_requests_py->>docs_community_release-process.rst: invokes
tests_test_requests_py->>docs_user_Aauthentication.rst: invokes
tests_test_requests_py->>docs__static_requests-sidebar.png: invokes
tests_test_requests_py->>docs__themes__gitignore: invokes
tests_test_requests_py->>ext_kr-compressed.png: invokes
tests_test_requests_py->>ext_psf-compressed.png: invokes
tests_test_requests_py->>ext_requests-logo-compressed.png: invokes
tests_test_requests_py->>ext_requests-logo.ai: invokes
tests_test_requests_py->>ext_requests-logo.png: invokes
tests_test_requests_py->>ext_requests-logo.svg: invokes
tests_test_requests_py->>ext_ss-compressed.png: invokes
tests_test_requests_py->>src_requests_adapters.py: invokes
tests_test_requests_py->>src_requests_api.py: invokes
tests_test_requests_py->>src_requests_auth.py: invokes
tests_test_requests_py->>src_requests_certs.py: invokes
tests_test_requests_py->>src_requests_compat.py: invokes
tests_test_requests_py->>src_requests_cookies.py: invokes
tests_test_requests_py->>src_requests_exceptions.py: invokes
tests_test_requests_py->>src_requests_help.py: invokes
tests_test_requests_py->>src_requests_hooks.py: invokes
tests_test_requests_py->>src_requests_models.py: invokes
tests_test_requests_py->>src_requests_packages.py: invokes
tests_test_requests_py->>src_requests_sessions.py: invokes
tests_test_requests_py->>src_requests_status_codes.py: invokes
tests_test_requests_py->>src_requests_structures.py: invokes
tests_test_requests_py->>src_requests_utils.py: invokes
tests_test_requests_py->>src_requests__internal_utils.py: invokes
tests_test_requests_py->>src_requests__init__.py: invokes
tests_test_requests_py->>src_requests__version__.py: invokes
tests_test_requests_py->>tests_compat.py: invokes
```

```
tests_test_requests_py->>tests_test_structures_py: invokes
tests_test_requests_py->>tests_test_utils_py: invokes
tests_test_requests_py->>tests_utils_py: invokes
tests_test_requests_py->>tests_certs_expired_Makefile: invokes
tests_test_requests_py->>tests_certs_expired_README_md: invokes
tests_test_requests_py->>tests_certs_expired_ca_ca-private_key: invokes
tests_test_requests_py->>tests_certs_expired_ca_ca_cnf: invokes
tests_test_requests_py->>tests_certs_expired_ca_ca_crt: invokes
tests_test_requests_py->>tests_certs_expired_ca_ca_srl: invokes
tests_test_requests_py->>tests_certs_expired_ca_Makefile: invokes
tests_test_requests_py->>tests_certs_expired_server_cert_cnf: invokes
tests_test_requests_py->>tests_certs_expired_server_Makefile: invokes
tests_test_requests_py->>tests_certs_expired_server_server_csr: invokes
tests_test_requests_py->>tests_certs_expired_server_server_key: invokes
tests_test_requests_py->>tests_certs_expired_server_server_pem: invokes
tests_test_requests_py->>tests_testserver_server_py: invokes
tests_test_requests_py->>tests_test_requests_py: invokes
src_requests_auth_py->>_git-blame-ignore-revs: invokes
src_requests_auth_py->>_gitignore: invokes
src_requests_auth_py->>_pre-commit-config_yaml: invokes
src_requests_auth_py->>_readthedocs_yaml: invokes
src_requests_auth_py->>requirements-dev_txt: invokes
src_requests_auth_py->>docs_requirements_txt: invokes
src_requests_auth_py->>docs_community_out-there_rst: invokes
src_requests_auth_py->>docs_community_recommended_rst: invokes
src_requests_auth_py->>docs_community_release-process_rst: invokes
src_requests_auth_py->>docs__static_requests-sidebar_png: invokes
src_requests_auth_py->>docs__themes__gitignore: invokes
src_requests_auth_py->>ext_kr-compressed_png: invokes
src_requests_auth_py->>ext_psf-compressed_png: invokes
src_requests_auth_py->>ext_requests-logo-compressed_png: invokes
src_requests_auth_py->>ext_requests-logo_ai: invokes
src_requests_auth_py->>ext_requests-logo_png: invokes
src_requests_auth_py->>ext_requests-logo_svg: invokes
src_requests_auth_py->>ext_ss-compressed_png: invokes
src_requests_auth_py->>src_requests_adapters_py: invokes
src_requests_auth_py->>src_requests_api_py: invokes
src_requests_auth_py->>src_requests_certs_py: invokes
src_requests_auth_py->>src_requests_compat_py: invokes
src_requests_auth_py->>src_requests_cookies_py: invokes
src_requests_auth_py->>src_requests_exceptions_py: invokes
src_requests_auth_py->>src_requests_help_py: invokes
src_requests_auth_py->>src_requests_hooks_py: invokes
src_requests_auth_py->>src_requests_models_py: invokes
src_requests_auth_py->>src_requests_packages_py: invokes
src_requests_auth_py->>src_requests_sessions_py: invokes
src_requests_auth_py->>src_requests_status_codes_py: invokes
src_requests_auth_py->>src_requests_structures_py: invokes
src_requests_auth_py->>src_requests_utils_py: invokes
src_requests_auth_py->>src_requests__internal_utils_py: invokes
src_requests_auth_py->>src_requests__init__py: invokes
src_requests_auth_py->>src_requests__version__py: invokes
src_requests_auth_py->>tests_compat_py: invokes
src_requests_auth_py->>tests_test_requests_py: invokes
```

```
src_requests_auth_py->>tests_test_structures_py: invokes
src_requests_auth_py->>tests_test_utils_py: invokes
src_requests_auth_py->>tests_utils_py: invokes
src_requests_auth_py->>tests_certs_expired_Makefile: invokes
src_requests_auth_py->>tests_certs_expired_README_md: invokes
src_requests_auth_py->>tests_certs_expired_ca_ca-private_key: invokes
src_requests_auth_py->>tests_certs_expired_ca_ca_cnf: invokes
src_requests_auth_py->>tests_certs_expired_ca_ca.crt: invokes
src_requests_auth_py->>tests_certs_expired_ca_ca_srl: invokes
src_requests_auth_py->>tests_certs_expired_ca_Makefile: invokes
src_requests_auth_py->>tests_certs_expired_server_cert_cnf: invokes
src_requests_auth_py->>tests_certs_expired_server_Makefile: invokes
src_requests_auth_py->>tests_certs_expired_server_server_csr: invokes
src_requests_auth_py->>tests_certs_expired_server_server_key: invokes
src_requests_auth_py->>tests_certs_expired_server_server_pem: invokes
src_requests_auth_py->>src_requests_auth_py: invokes
src_requests_utils_py->>_git-blame-ignore-revs: invokes
src_requests_utils_py->>_gitignore: invokes
src_requests_utils_py->>_pre-commit-config.yaml: invokes
src_requests_utils_py->>_readthedocs.yaml: invokes
src_requests_utils_py->>requirements-dev.txt: invokes
src_requests_utils_py->>docs_requirements.txt: invokes
src_requests_utils_py->>docs_community_out-there_rst: invokes
src_requests_utils_py->>docs_community_recommended_rst: invokes
src_requests_utils_py->>docs_community_release-process_rst: invokes
src_requests_utils_py->>docs_user_authentication_rst: invokes
src_requests_utils_py->>docs__static_requests-sidebar_png: invokes
src_requests_utils_py->>docs__themes__gitignore: invokes
src_requests_utils_py->>ext_kr-compressed_png: invokes
src_requests_utils_py->>ext_psf-compressed_png: invokes
src_requests_utils_py->>ext_requests-logo-compressed_png: invokes
src_requests_utils_py->>ext_requests-logo_ai: invokes
src_requests_utils_py->>ext_requests-logo_png: invokes
src_requests_utils_py->>ext_requests-logo_svg: invokes
src_requests_utils_py->>ext_ss-compressed_png: invokes
src_requests_utils_py->>src_requests_adapters_py: invokes
src_requests_utils_py->>src_requests_api_py: invokes
src_requests_utils_py->>src_requests_auth_py: invokes
src_requests_utils_py->>src_requests_certs_py: invokes
src_requests_utils_py->>src_requests_compat_py: invokes
src_requests_utils_py->>src_requests_cookies_py: invokes
src_requests_utils_py->>src_requests_exceptions_py: invokes
src_requests_utils_py->>src_requests_help_py: invokes
src_requests_utils_py->>src_requests_hooks_py: invokes
src_requests_utils_py->>src_requests_models_py: invokes
src_requests_utils_py->>src_requests_packages_py: invokes
src_requests_utils_py->>src_requests_sessions_py: invokes
src_requests_utils_py->>src_requests_status_codes_py: invokes
src_requests_utils_py->>src_requests_structures_py: invokes
src_requests_utils_py->>src_requests__internal_utils_py: invokes
src_requests_utils_py->>src_requests__init__py: invokes
src_requests_utils_py->>src_requests__version__py: invokes
src_requests_utils_py->>tests_compat_py: invokes
src_requests_utils_py->>tests_test_requests_py: invokes
```

```
src_requests_utils_py->>tests_test_structures_py: invokes
src_requests_utils_py->>tests_certs_expired_Makefile: invokes
src_requests_utils_py->>tests_certs_expired_README_md: invokes
src_requests_utils_py->>tests_certs_expired_ca_ca-private_key: invokes
src_requests_utils_py->>tests_certs_expired_ca_ca_cnf: invokes
src_requests_utils_py->>tests_certs_expired_ca_ca_crt: invokes
src_requests_utils_py->>tests_certs_expired_ca_ca_srl: invokes
src_requests_utils_py->>tests_certs_expired_ca_Makefile: invokes
src_requests_utils_py->>tests_certs_expired_server_cert_cnf: invokes
src_requests_utils_py->>tests_certs_expired_server_Makefile: invokes
src_requests_utils_py->>tests_certs_expired_server_server_csr: invokes
src_requests_utils_py->>tests_certs_expired_server_server_key: invokes
src_requests_utils_py->>tests_certs_expired_server_server_pem: invokes
src_requests_utils_py->>src_requests_utils_py: invokes
src_requests__internal_utils_py->>git-blame-ignore-revs: invokes
src_requests__internal_utils_py->>gitignore: invokes
src_requests__internal_utils_py->>pre-commit-config_yaml: invokes
src_requests__internal_utils_py->>readthedocs_yaml: invokes
src_requests__internal_utils_py->>requirements-dev_txt: invokes
src_requests__internal_utils_py->>docs_requirements_txt: invokes
src_requests__internal_utils_py->>docs_community_out-there_rst: invokes
src_requests__internal_utils_py->>docs_community_recommended_rst: invokes
src_requests__internal_utils_py->>docs_community_release-process_rst: invokes
src_requests__internal_utils_py->>docs__static_requests-sidebar_png: invokes
src_requests__internal_utils_py->>docs__themes__gitignore: invokes
src_requests__internal_utils_py->>ext_kr-compressed_png: invokes
src_requests__internal_utils_py->>ext_psf-compressed_png: invokes
src_requests__internal_utils_py->>ext_requests-logo-compressed_png: invokes
src_requests__internal_utils_py->>ext_requests-logo_ai: invokes
src_requests__internal_utils_py->>ext_requests-logo_png: invokes
src_requests__internal_utils_py->>ext_requests-logo_svg: invokes
src_requests__internal_utils_py->>ext_ss-compressed_png: invokes
src_requests__internal_utils_py->>src_requests_adapters_py: invokes
src_requests__internal_utils_py->>src_requests_api_py: invokes
src_requests__internal_utils_py->>src_requests_auth_py: invokes
src_requests__internal_utils_py->>src_requests_certs_py: invokes
src_requests__internal_utils_py->>src_requests_compat_py: invokes
src_requests__internal_utils_py->>src_requests_cookies_py: invokes
src_requests__internal_utils_py->>src_requests_exceptions_py: invokes
src_requests__internal_utils_py->>src_requests_help_py: invokes
src_requests__internal_utils_py->>src_requests_hooks_py: invokes
src_requests__internal_utils_py->>src_requests_models_py: invokes
src_requests__internal_utils_py->>src_requests_packages_py: invokes
src_requests__internal_utils_py->>src_requests_sessions_py: invokes
src_requests__internal_utils_py->>src_requests_status_codes_py: invokes
src_requests__internal_utils_py->>src_requests_structures_py: invokes
src_requests__internal_utils_py->>src_requests_utils_py: invokes
src_requests__internal_utils_py->>src_requests__init__py: invokes
src_requests__internal_utils_py->>src_requests__version__py: invokes
src_requests__internal_utils_py->>tests_compat_py: invokes
src_requests__internal_utils_py->>tests_test_requests_py: invokes
src_requests__internal_utils_py->>tests_test_structures_py: invokes
src_requests__internal_utils_py->>tests_certs_expired_Makefile: invokes
src_requests__internal_utils_py->>tests_certs_expired_README_md: invokes
```

```
src_requests__internal_utils_py->>tests_certs_expired_ca_ca-private_key: invokes
src_requests__internal_utils_py->>tests_certs_expired_ca_ca_cnf: invokes
src_requests__internal_utils_py->>tests_certs_expired_ca_ca_crt: invokes
src_requests__internal_utils_py->>tests_certs_expired_ca_ca_srl: invokes
src_requests__internal_utils_py->>tests_certs_expired_ca_Makefile: invokes
src_requests__internal_utils_py->>tests_certs_expired_server_cert_cnf: invokes
src_requests__internal_utils_py->>tests_certs_expired_server_Makefile: invokes
src_requests__internal_utils_py->>tests_certs_expired_server_server_csr: invokes
src_requests__internal_utils_py->>tests_certs_expired_server_server_key: invokes
src_requests__internal_utils_py->>tests_certs_expired_server_server_pem: invokes
src_requests__internal_utils_py->>src_requests__internal_utils_py: invokes
tests_test_lowlevel_py->>docs__static_requests-sidebar_png: invokes
tests_test_lowlevel_py->>ext_requests-logo-compressed_png: invokes
tests_test_lowlevel_py->>ext_requests-logo_ai: invokes
tests_test_lowlevel_py->>ext_requests-logo_png: invokes
tests_test_lowlevel_py->>ext_requests-logo_svg: invokes
tests_test_lowlevel_py->>src_requests_adapters_py: invokes
tests_test_lowlevel_py->>src_requests_api_py: invokes
tests_test_lowlevel_py->>src_requests_auth_py: invokes
tests_test_lowlevel_py->>src_requests_certs_py: invokes
tests_test_lowlevel_py->>src_requests_compat_py: invokes
tests_test_lowlevel_py->>src_requests_cookies_py: invokes
tests_test_lowlevel_py->>src_requests_exceptions_py: invokes
tests_test_lowlevel_py->>src_requests_help_py: invokes
tests_test_lowlevel_py->>src_requests_hooks_py: invokes
tests_test_lowlevel_py->>src_requests_models_py: invokes
tests_test_lowlevel_py->>src_requests_packages_py: invokes
tests_test_lowlevel_py->>src_requests_sessions_py: invokes
tests_test_lowlevel_py->>src_requests_status_codes_py: invokes
tests_test_lowlevel_py->>src_requests_structures_py: invokes
tests_test_lowlevel_py->>src_requests_utils_py: invokes
tests_test_lowlevel_py->>src_requests__internal_utils_py: invokes
tests_test_lowlevel_py->>src_requests__init__py: invokes
tests_test_lowlevel_py->>src_requests__version__py: invokes
tests_test_lowlevel_py->>tests_test_requests_py: invokes
tests_test_lowlevel_py->>tests_test_utils_py: invokes
tests_test_lowlevel_py->>tests_utils_py: invokes
tests_test_lowlevel_py->>tests_testserver_server_py: invokes
tests_test_utils_py->>docs_user_authentication_rst: invokes
tests_test_utils_py->>docs__static_requests-sidebar_png: invokes
tests_test_utils_py->>ext_requests-logo-compressed_png: invokes
tests_test_utils_py->>ext_requests-logo_ai: invokes
tests_test_utils_py->>ext_requests-logo_png: invokes
tests_test_utils_py->>ext_requests-logo_svg: invokes
tests_test_utils_py->>src_requests_adapters_py: invokes
tests_test_utils_py->>src_requests_api_py: invokes
tests_test_utils_py->>src_requests_auth_py: invokes
tests_test_utils_py->>src_requests_certs_py: invokes
tests_test_utils_py->>src_requests_compat_py: invokes
tests_test_utils_py->>src_requests_cookies_py: invokes
tests_test_utils_py->>src_requests_exceptions_py: invokes
tests_test_utils_py->>src_requests_help_py: invokes
tests_test_utils_py->>src_requests_hooks_py: invokes
tests_test_utils_py->>src_requests_models_py: invokes
```

```
tests_test_utils_py->>src_requests_packages_py: invokes
tests_test_utils_py->>src_requests_sessions_py: invokes
tests_test_utils_py->>src_requests_status_codes_py: invokes
tests_test_utils_py->>src_requests_structures_py: invokes
tests_test_utils_py->>src_requests_utils_py: invokes
tests_test_utils_py->>src_requests__internal_utils_py: invokes
tests_test_utils_py->>src_requests__init__py: invokes
tests_test_utils_py->>src_requests__version__py: invokes
tests_test_utils_py->>tests_compat_py: invokes
tests_test_utils_py->>tests_test_requests_py: invokes
tests_test_testserver_py->>docs__static_requests-sidebar_png: invokes
tests_test_testserver_py->>ext_requests-logo-compressed_png: invokes
tests_test_testserver_py->>ext_requests-logo_ai: invokes
tests_test_testserver_py->>ext_requests-logo_png: invokes
tests_test_testserver_py->>ext_requests-logo_svg: invokes
tests_test_testserver_py->>src_requests_adapters_py: invokes
tests_test_testserver_py->>src_requests_api_py: invokes
tests_test_testserver_py->>src_requests_auth_py: invokes
tests_test_testserver_py->>src_requests_certs_py: invokes
tests_test_testserver_py->>src_requests_compat_py: invokes
tests_test_testserver_py->>src_requests_cookies_py: invokes
tests_test_testserver_py->>src_requests_exceptions_py: invokes
tests_test_testserver_py->>src_requests_help_py: invokes
tests_test_testserver_py->>src_requests_hooks_py: invokes
tests_test_testserver_py->>src_requests_models_py: invokes
tests_test_testserver_py->>src_requests_packages_py: invokes
tests_test_testserver_py->>src_requests_sessions_py: invokes
tests_test_testserver_py->>src_requests_status_codes_py: invokes
tests_test_testserver_py->>src_requests_structures_py: invokes
tests_test_testserver_py->>src_requests_utils_py: invokes
tests_test_testserver_py->>src_requests__internal_utils_py: invokes
tests_test_testserver_py->>src_requests__init__py: invokes
tests_test_testserver_py->>src_requests__version__py: invokes
tests_test_testserver_py->>tests_test_requests_py: invokes
tests_test_testserver_py->>tests_testserver_server_py: invokes
docs_conf_py->>docs__static_requests-sidebar_png: invokes
docs_conf_py->>ext_requests-logo-compressed_png: invokes
docs_conf_py->>ext_requests-logo_ai: invokes
docs_conf_py->>ext_requests-logo_png: invokes
docs_conf_py->>ext_requests-logo_svg: invokes
docs_conf_py->>src_requests_adapters_py: invokes
docs_conf_py->>src_requests_api_py: invokes
docs_conf_py->>src_requests_auth_py: invokes
docs_conf_py->>src_requests_certs_py: invokes
docs_conf_py->>src_requests_compat_py: invokes
docs_conf_py->>src_requests_cookies_py: invokes
docs_conf_py->>src_requests_exceptions_py: invokes
docs_conf_py->>src_requests_help_py: invokes
docs_conf_py->>src_requests_hooks_py: invokes
docs_conf_py->>src_requests_models_py: invokes
docs_conf_py->>src_requests_packages_py: invokes
docs_conf_py->>src_requests_sessions_py: invokes
docs_conf_py->>src_requests_status_codes_py: invokes
docs_conf_py->>src_requests_structures_py: invokes
```

```
docs_conf_py->>src_requests_utils_py: invokes
docs_conf_py->>src_requests__internal_utils_py: invokes
docs_conf_py->>src_requests__init__py: invokes
docs_conf_py->>src_requests__version__py: invokes
docs_conf_py->>tests_test_requests_py: invokes
tests_test_hooks_py->>docs__static_requests-sidebar_png: invokes
tests_test_hooks_py->>ext_requests-logo-compressed_png: invokes
tests_test_hooks_py->>ext_requests-logo_ai: invokes
tests_test_hooks_py->>ext_requests-logo_png: invokes
tests_test_hooks_py->>ext_requests-logo_svg: invokes
tests_test_hooks_py->>src_requests_adapters_py: invokes
tests_test_hooks_py->>src_requests_api_py: invokes
tests_test_hooks_py->>src_requests_auth_py: invokes
tests_test_hooks_py->>src_requests_certs_py: invokes
tests_test_hooks_py->>src_requests_compat_py: invokes
tests_test_hooks_py->>src_requests_cookies_py: invokes
tests_test_hooks_py->>src_requests_exceptions_py: invokes
tests_test_hooks_py->>src_requests_help_py: invokes
tests_test_hooks_py->>src_requests_hooks_py: invokes
tests_test_hooks_py->>src_requests_models_py: invokes
tests_test_hooks_py->>src_requests_packages_py: invokes
tests_test_hooks_py->>src_requests_sessions_py: invokes
tests_test_hooks_py->>src_requests_status_codes_py: invokes
tests_test_hooks_py->>src_requests_structures_py: invokes
tests_test_hooks_py->>src_requests_utils_py: invokes
tests_test_hooks_py->>src_requests__internal_utils_py: invokes
tests_test_hooks_py->>src_requests__init__py: invokes
tests_test_hooks_py->>src_requests__version__py: invokes
tests_test_hooks_py->>tests_test_requests_py: invokes
tests_test_packages_py->>docs__static_requests-sidebar_png: invokes
tests_test_packages_py->>ext_requests-logo-compressed_png: invokes
tests_test_packages_py->>ext_requests-logo_ai: invokes
tests_test_packages_py->>ext_requests-logo_png: invokes
tests_test_packages_py->>ext_requests-logo_svg: invokes
tests_test_packages_py->>src_requests_adapters_py: invokes
tests_test_packages_py->>src_requests_api_py: invokes
tests_test_packages_py->>src_requests_auth_py: invokes
tests_test_packages_py->>src_requests_certs_py: invokes
tests_test_packages_py->>src_requests_compat_py: invokes
tests_test_packages_py->>src_requests_cookies_py: invokes
tests_test_packages_py->>src_requests_exceptions_py: invokes
tests_test_packages_py->>src_requests_help_py: invokes
tests_test_packages_py->>src_requests_hooks_py: invokes
tests_test_packages_py->>src_requests_models_py: invokes
tests_test_packages_py->>src_requests_packages_py: invokes
tests_test_packages_py->>src_requests_sessions_py: invokes
tests_test_packages_py->>src_requests_status_codes_py: invokes
tests_test_packages_py->>src_requests_structures_py: invokes
tests_test_packages_py->>src_requests_utils_py: invokes
tests_test_packages_py->>src_requests__internal_utils_py: invokes
tests_test_packages_py->>src_requests__init__py: invokes
tests_test_packages_py->>src_requests__version__py: invokes
tests_test_packages_py->>tests_test_requests_py: invokes
src_requests_sessions_py->>docs_dev_authors_rst: invokes
```



```

src_requests_sessions_py->>docs_user_authentication_rst: invokes
src_requests_sessions_py->>src_requests_adapters_py: invokes
src_requests_sessions_py->>src_requests_auth_py: invokes
src_requests_sessions_py->>src_requests_compat_py: invokes
src_requests_sessions_py->>src_requests_cookies_py: invokes
src_requests_sessions_py->>src_requests_exceptions_py: invokes
src_requests_sessions_py->>src_requests_hooks_py: invokes
src_requests_sessions_py->>src_requests_models_py: invokes
src_requests_sessions_py->>src_requests_status_codes_py: invokes
src_requests_sessions_py->>src_requests_structures_py: invokes
src_requests_sessions_py->>src_requests_utils_py: invokes
src_requests_sessions_py->>src_requests__internal_utils_py: invokes
src_requests_sessions_py->>tests_compat_py: invokes
src_requests_sessions_py->>tests_test_adapters_py: invokes
src_requests_sessions_py->>tests_test_hooks_py: invokes
src_requests_sessions_py->>tests_test_structures_py: invokes
src_requests_sessions_py->>tests_test_utils_py: invokes
src_requests_sessions_py->>tests_utils_py: invokes
src_requests_models_py->>docs_dev_authors_rst: invokes
src_requests_models_py->>docs_user_authentication_rst: invokes
src_requests_models_py->>src_requests_auth_py: invokes
src_requests_models_py->>src_requests_compat_py: invokes
src_requests_models_py->>src_requests_cookies_py: invokes
src_requests_models_py->>src_requests_exceptions_py: invokes
src_requests_models_py->>src_requests_hooks_py: invokes
src_requests_models_py->>src_requests_sessions_py: invokes
src_requests_models_py->>src_requests_status_codes_py: invokes
src_requests_models_py->>src_requests_structures_py: invokes
src_requests_models_py->>src_requests_utils_py: invokes
src_requests_models_py->>src_requests__internal_utils_py: invokes
src_requests_models_py->>src_requests__version__py: invokes
src_requests_models_py->>tests_compat_py: invokes
src_requests_models_py->>tests_test_hooks_py: invokes
src_requests_models_py->>tests_test_structures_py: invokes
src_requests_models_py->>tests_test_utils_py: invokes
src_requests_models_py->>tests_utils_py: invokes

```

03. Module Deep-Dive

3.1 tests/test_requests.py

The `tests/test_requests.py` module executes a comprehensive suite of tests to validate the functionality of the `requests` library. The module's execution logic is as follows:

1. **Test Class Execution:** The module contains multiple test classes, including `TestRequests`, `TestCaseInsensitiveDict`, `TestMorselToCookieExpires`, `TestMorselToCookieMaxAge`, `TestTimeout`, `RedirectSession`, and `TestPreparingURLs`. Each test class is instantiated and executed, running its respective test methods.
2. **Test Method Execution:** Within each test class, various test methods are executed. These methods test specific aspects of the `requests` library, such as request preparation, data encoding, and cookie

handling.

3. **Function Execution:** The module also contains standalone test functions, including `test_json_encodes_as_bytes`, `test_requests_are_updated_each_time`, `test_proxy_env_vars_override_default`, `test_data_argument_accepts_tuples`, `test_prepared_copy`, `test_urllib3_retries`, `test_urllib3_pool_connection_closed`, `test_content_length_for_bytes_data`, `test_content_length_for_string_data_counts_bytes`, and `test_json_decode_errors_are_serializable_deserializable`. These functions are executed independently and test specific features of the `requests` library.
4. **Data Processing:** The module processes various types of data, including JSON, bytes, and strings. Test methods and functions validate the encoding, decoding, and handling of these data types, ensuring that the `requests` library behaves correctly.
5. **Assertion and Validation:** Throughout the execution of the test methods and functions, assertions and validations are performed to ensure that the expected behavior of the `requests` library is observed. Any deviations from the expected behavior result in test failures.

In terms of data processing, the module's execution logic can be summarized as follows:

- JSON data is encoded and decoded using the `json` library, and the resulting data is validated to ensure correct serialization and deserialization.
- Bytes data is processed and validated to ensure correct handling of content lengths and encoding.
- String data is processed and validated to ensure correct handling of content lengths and encoding.
- Tuples of strings are accepted as data arguments and properly encoded.
- The `requests` library's retry mechanism is tested to ensure correct behavior in the presence of connection closures and retries.

Overall, the `tests/test_requests.py` module provides a comprehensive suite of tests to validate the functionality of the `requests` library, ensuring that it behaves correctly and processes data as expected.

Component	Type	Responsibility
TestRequests	Class	The Class <code>TestRequests</code> is a specialized component within the requests system, responsible for encapsulating and executing test-related requests. Its primary responsibility is to manage the creation, execution, and verification of test requests, ensuring that the system's functionality is properly validated.

The specific responsibilities of the `TestRequests` Class include:

1. Encapsulating test request data: The Class `TestRequests` encapsulates the data required for test requests, such as test case identifiers, input parameters, and expected results.
2. Providing a standardized interface: The Class `TestRequests` provides a standardized interface for creating and executing test requests, allowing for consistent and repeatable testing.
3. Executing test requests: The Class `TestRequests` is responsible for executing the test requests, which involves sending the request to the relevant system components and retrieving the response.
4. Verifying test results: The Class `TestRequests` verifies the test results, comparing the actual response with the expected results to determine whether the test has passed or failed.

5. Reporting test outcomes: The Class `TestRequests` reports the test outcomes, providing detailed information on the test results, including any errors or exceptions that occurred during execution.

By assuming these responsibilities, the `TestRequests` Class plays a crucial role in ensuring the reliability and stability of the requests system, enabling developers to identify and rectify defects, and ultimately guaranteeing the delivery of high-quality software. | | `TestCaseInsensitiveDict` | Class | The `TestCaseInsensitiveDict` class in the requests system provides a dictionary-like data structure that treats keys in a case-insensitive manner. This class is specifically designed to handle HTTP headers, where the keys are case-insensitive according to the HTTP specification.

The primary responsibility of `TestCaseInsensitiveDict` is to ensure that keys are compared and accessed in a case-insensitive manner, allowing for consistent and accurate handling of HTTP headers. This class achieves this by converting all keys to lowercase internally, while preserving the original casing of the keys when returning them.

By using `TestCaseInsensitiveDict`, the requests system can correctly handle HTTP headers with varying casing, ensuring that headers such as "Content-Type" and "content-type" are treated as equivalent. This class plays a crucial role in maintaining the integrity and consistency of HTTP header processing within the requests system. | | `TestMorselToCookieExpires` | Class | Tests for morsel_to_cookie when morsel contains expires. | | `TestMorselToCookieMaxAge` | Class | Tests for morsel_to_cookie when morsel contains max-age. | | `TestTimeout` | Class | The `TestTimeout` class is responsible for implementing a timeout mechanism for test cases in the requests system. Its primary function is to interrupt and terminate test cases that exceed a predetermined time limit, preventing them from running indefinitely and consuming system resources.

This class specifically handles the timing out of test cases by utilizing a timer that monitors the execution time of each test case. When the allotted time is exhausted, the `TestTimeout` class intervenes, halting the test case and reporting a timeout error. This ensures that the test suite remains efficient and responsive, even in the presence of time-consuming or non-terminating test cases.

By providing this functionality, the `TestTimeout` class plays a crucial role in maintaining the overall performance and reliability of the requests system's testing infrastructure. | | `RedirectSession` | Class | The `RedirectSession` class is responsible for managing the HTTP redirect lifecycle, ensuring seamless transitions between requests and maintaining the integrity of session data throughout the redirect process.

Its specific responsibilities include:

1. Capturing the current request state: The `RedirectSession` class captures the current request's state, including parameters, headers, and any relevant session data, to preserve the original request context.
2. Storing session data: It stores the captured session data securely, using a reliable storage mechanism, to ensure that the data remains accessible throughout the redirect process.
3. Generating redirect URLs: The `RedirectSession` class generates redirect URLs, incorporating the necessary parameters and session identifiers, to facilitate the redirect process.
4. Validating redirect requests: Upon receiving a redirect request, it validates the request's authenticity and integrity, ensuring that the request is legitimate and has not been tampered with.
5. Restoring session data: After validating the redirect request, the `RedirectSession` class restores the original session data, allowing the application to resume processing the request as if the redirect had

not occurred.

By assuming these responsibilities, the `RedirectSession` class ensures a transparent and efficient redirect process, minimizing disruptions to the user experience and maintaining the security and integrity of session data. || `test_json_encodes_as_bytes` | Function | The `test_json_encodes_as_bytes` function is responsible for verifying that the JSON encoding functionality within the requests system correctly serializes JSON data into bytes. This function tests the system's ability to properly handle JSON encoding, ensuring that the resulting encoded data is in bytes format, which is the expected output for transmission over a network.

In the context of the requests system, this function plays a crucial role in ensuring the integrity of JSON data sent in HTTP requests. By confirming that JSON encoding produces bytes, the function helps guarantee that the system can correctly transmit JSON data to servers, which is essential for maintaining interoperability and preventing data corruption during transmission.

The specific responsibilities of the `test_json_encodes_as_bytes` function include:

1. Verifying that the JSON encoding process produces a bytes object as output.
2. Confirming that the resulting bytes object accurately represents the original JSON data.
3. Ensuring that the system correctly handles edge cases, such as encoding nested JSON structures or dealing with special characters.

By fulfilling these responsibilities, the `test_json_encodes_as_bytes` function contributes to the overall reliability and robustness of the requests system, providing confidence that JSON data is correctly encoded and transmitted as intended. || `test_requests_are_updated_each_time` | Function | The function `test_requests_are_updated_each_time` is responsible for verifying that the requests in the system are successfully updated each time a modification is made, ensuring data consistency and integrity.

This function validates the request update mechanism by simulating multiple update scenarios and checking the outcome against expected results. Specifically, it tests whether the system accurately reflects changes made to existing requests, including updates to request status, data, and other relevant attributes.

By exercising this function, the system guarantees that request updates are propagated correctly, and any issues or discrepancies are identified and addressed in a timely manner, thereby maintaining the overall reliability and accuracy of the requests system. || `test_proxy_env_vars_override_default` | Function | The function `test_proxy_env_vars_override_default` is responsible for verifying that environment variables override the default proxy settings in the requests system.

This function tests the priority of environment variables in setting proxy configurations, ensuring that they take precedence over the default settings. It checks that when environment variables for proxy settings are set, they successfully override the default proxy configurations used by the requests system.

In essence, this function guarantees that environment variables, such as `http_proxy` and `https_proxy`, can be used to customize the proxy settings for HTTP requests, providing a mechanism for dynamic configuration and flexibility in the requests system.

Here is a high-level representation of how the test function operates:

1. Set environment variables for proxy settings (e.g., `http_proxy` and `https_proxy`).
2. Initialize a requests session with default proxy settings.
3. Use the session to send an HTTP request.

4. Verify that the request uses the proxy settings specified by the environment variables, rather than the default settings.

By performing this test, `test_proxy_env_vars_override_default` ensures that the requests system behaves as expected when environment variables are used to override default proxy settings, providing a robust and reliable mechanism for configuring proxy connections. || `test_data_argument_accepts_tuples` | Function | Ensure that the data argument will accept tuples of strings and properly encode them. || `test_prepared_copy` | Function | The function `test_prepared_copy` is a unit test within the requests system, specifically responsible for verifying the correctness of the `prepared_copy` method.

This function tests that the `prepared_copy` method accurately creates a copy of a `PreparedRequest` object, ensuring that all attributes are duplicated without modifying the original request. The test validates that the copied request contains the same properties as the original request, including but not limited to:

- Headers
- Query parameters
- Body data
- URL
- Method

By exercising the `prepared_copy` method under various scenarios, `test_prepared_copy` ensures that the method behaves as expected, providing a robust and reliable mechanism for creating duplicate requests within the requests system. This test plays a critical role in maintaining the integrity and consistency of the requests library, allowing developers to confidently utilize the `prepared_copy` method in their applications. || `test_urllib3_retries` | Function | The function `test_urllib3_retries` is a test case within the requests system's test suite, specifically responsible for verifying the correct behavior of the urllib3 retry mechanism.

This function tests the ability of the requests library to retry failed HTTP requests using the urllib3 library, which is the underlying HTTP client used by requests. The test case exercises the retry logic by simulating various failure scenarios, such as connection timeouts, DNS resolution failures, and HTTP server errors, to ensure that the library correctly retries the request the specified number of times before raising an exception.

In essence, `test_urllib3_retries` validates that the urllib3 retry mechanism is properly integrated with the requests library and functions as expected under various error conditions, thereby guaranteeing the reliability and fault tolerance of the requests system. || `test_urllib3_pool_connection_closed` | Function | The function `test_urllib3_pool_connection_closed` is responsible for verifying that the connection pool implemented by urllib3 correctly closes connections when they are no longer needed.

This test function specifically checks the behavior of the `PoolManager` class, which is responsible for managing a pool of connections to a given host. When a connection is closed, the `PoolManager` should remove it from the pool to prevent further use and allow the underlying socket to be released.

By exercising this specific scenario, `test_urllib3_pool_connection_closed` ensures that the requests system does not leak connections, which can lead to resource exhaustion and other issues over time. This test provides essential validation of the requests library's integration with urllib3 and helps maintain the overall reliability and performance of the system. || `TestPreparingURLs` | Class | The Class `TestPreparingURLs` is responsible for constructing and preparing URLs for test requests, ensuring that all necessary parameters, authentication tokens, and other required elements are properly incorporated and formatted according to the system's specifications.

This class plays a critical role in the requests system by:

1. Validating input parameters to guarantee correctness and consistency.
2. Encoding and appending query parameters to the base URL.
3. Injecting authentication tokens, such as API keys or OAuth credentials, into the URL or request headers.
4. Formatting the URL according to the system's routing rules and conventions.
5. Providing a standardized interface for generating test URLs, promoting code reusability and reducing duplication.

By encapsulating the logic for preparing test URLs within the `TestPreparingURLs` Class, the system ensures that all test requests are properly configured and formatted, reducing the likelihood of errors and inconsistencies. This class serves as a key component in the requests system's architecture, promoting reliability, maintainability, and scalability. || `test_content_length_for_bytes_data` | Function | The `test_content_length_for_bytes_data` function is responsible for verifying that the Content-Length header is correctly set in HTTP requests when the request body contains bytes data.

In the requests system, this function tests the logic for calculating the Content-Length header value when the request body is a bytes object. It ensures that the Content-Length header is set to the correct byte count, matching the size of the bytes data being sent in the request body.

By validating the Content-Length header calculation for bytes data, this function helps guarantee that HTTP requests are properly formatted and can be accurately processed by servers, preventing potential issues such as request truncation or incorrect payload parsing. ||

`test_content_length_for_string_data_counts_bytes` | Function | The function `test_content_length_for_string_data_counts_bytes` is responsible for verifying that the `Content-Length` header in HTTP requests accurately reflects the byte count of string data, rather than the character count.

This function tests the behavior of the requests system when sending string data, ensuring that the system correctly calculates the byte count of the data, taking into account the encoding used, and sets the `Content-Length` header accordingly.

Specifically, this function is designed to prevent issues that may arise when sending string data that contains non-ASCII characters, which may be encoded using multiple bytes per character. By verifying that the `Content-Length` header accurately reflects the byte count, this function ensures that the requests system correctly handles string data and avoids potential errors or truncation of data during transmission.

In essence, `test_content_length_for_string_data_counts_bytes` is a quality assurance mechanism that validates the requests system's handling of string data and its compliance with HTTP protocol requirements.

The function's specific responsibilities include:

1. Sending a request with string data containing non-ASCII characters.
2. Verifying that the `Content-Length` header is set correctly, reflecting the byte count of the data rather than the character count.
3. Ensuring that the requests system correctly handles string data encoding and calculates the byte count accurately.

By fulfilling these responsibilities, `test_content_length_for_string_data_counts_bytes` ensures the reliability and correctness of the requests system when handling string data. ||

`test_json_decode_errors_are_serializable_deserializable` | Function | The `test_json_decode_errors_are_serializable_deserializable` function is responsible for verifying that JSON decode errors encountered by the requests system are both serializable and deserializable.

This function tests the requests system's ability to handle JSON decode errors by intentionally inducing such errors and then serializing and deserializing them. The goal is to ensure that these errors can be properly represented, stored, and reconstructed without losing any essential information.

Specifically, this function checks that JSON decode errors are correctly serialized into a format that can be written to a file, sent over a network, or stored in a database. It then verifies that these serialized errors can be deserialized back into their original form, allowing the requests system to accurately recreate and handle the original error.

By performing this test, the `test_json_decode_errors_are_serializable_deserializable` function guarantees that the requests system's error handling mechanisms are robust and reliable, enabling the system to maintain its integrity and functionality even in the presence of JSON decode errors. |

3.2 `src/requests/auth.py`

The `src/requests/auth.py` module executes the following logic to process authentication data:

1. The `_basic_auth_str` function generates a Basic Auth string from the provided username and password. It combines the username and password with a colon, encodes the resulting string in Base64, and prefixes it with the string "Basic ".
2. The `AuthBase` class serves as a base class for all authentication implementations. It defines a basic structure for authentication classes, including an `__init__` method for initialization and a `__call__` method for attaching authentication data to a Request object.
3. The `HTTPBasicAuth` class inherits from `AuthBase` and implements Basic HTTP Authentication. When instantiated, it stores the provided username and password. Upon being called, it generates a Basic Auth string using the `_basic_auth_str` function and attaches it to the Request object's headers under the "Authorization" key.
4. The `HTTPProxyAuth` class also inherits from `AuthBase` and implements HTTP Proxy Authentication. Its execution logic is similar to `HTTPBasicAuth`, but it attaches the authentication data to the Request object's headers under the "Proxy-Authorization" key.
5. The `HTTPDigestAuth` class implements HTTP Digest Authentication. When instantiated, it stores the provided username and password. Upon being called, it generates a unique nonce and attaches the necessary authentication data to the Request object's headers under the "Authorization" key. This data includes the username, realm, nonce, and response, which is generated using the MD5 hash algorithm.

In summary, this module processes authentication data by generating the required authentication strings or headers for different authentication schemes (Basic Auth, Proxy Auth, and Digest Auth) and attaching them to the Request object. The specific execution logic depends on the chosen authentication class.

Component	Type	Responsibility
<code>_basic_auth_str</code>	Function	Returns a Basic Auth string.

Component	Type	Responsibility
<code>AuthBase</code>	Class	Base class that all auth implementations derive from
<code>HTTPBasicAuth</code>	Class	Attaches HTTP Basic Authentication to the given Request object.
<code>HTTPProxyAuth</code>	Class	Attaches HTTP Proxy Authentication to a given Request object.
<code>HTTPDigestAuth</code>	Class	Attaches HTTP Digest Authentication to the given Request object.

3.3 `src/requests/utils.py`

The `src/requests/utils.py` module contains a collection of utility functions that facilitate data processing for HTTP requests. Here's a breakdown of the execution logic for each function:

Data Conversion and Validation

1. `dict_to_sequence`: Converts a dictionary into an internal sequence dictionary update.
2. `from_key_val_list` and `to_key_val_list`: These functions convert between a dictionary and a list of tuples, ensuring that the input data can be represented as a dictionary.
3. `check_header_validity` and `_validate_header_part`: Validate header parts to ensure they don't contain leading whitespace, reserved characters, or return characters.

Data Extraction and Parsing

1. `get_netrc_auth`: Extracts the Requests tuple auth for a given URL from netrc.
2. `guess_filename`: Attempts to guess the filename of a given object.
3. `extract_zipped_paths`: Replaces nonexistent paths that resemble a zip archive member with the location of an extracted copy.
4. `parse_list_header` and `parse_dict_header`: Parse lists and dictionaries from header strings according to RFC 2068 Section 2.
5. `unquote_header_value`: Unquotes a header value using a reversal of the `quote_header_value` function.
6. `get_encodings_from_content` and `get_encoding_from_headers`: Extract encodings from a given content string or HTTP header dictionary.

Data Transformation and Manipulation

1. `iter_slices`: Iterates over slices of a string.
2. `stream_decode_response_unicode`: Stream decodes an iterator.
3. `get_unicode_from_response`: Returns the requested content in Unicode, attempting to use the charset from the content-type header or falling back to replacing all Unicode characters.
4. `unquote_unreserved`: Un-escapes any percent-escape sequences in a URI that are unreserved characters.
5. `requote_uri`: Re-quotes a given URI to ensure consistent quoting.

Network and IP Address Utilities

1. `address_in_network`: Checks if an IP address belongs to a network subnet.
2. `dotted_netmask`: Converts a mask from /xx format to xxx.xxx.xxx.xxx format.
3. `is_ipv4_address` and `is_valid_cidr`: Validate IP addresses and CIDR formats.

Environment and Proxy Utilities

- 1. `set_environ`: Sets an environment variable to a given value, saving the previous value and restoring it later.
- 2. `should_bypass_proxies`: Determines whether to bypass proxies or not.
- 3. `get_environ_proxies`: Returns a dictionary of environment proxies.
- 4. `select_proxy` and `resolve_proxies`: Select a proxy for a given URL and resolve proxy information from a request and configuration input.

Miscellaneous Utilities

- 1. `default_user_agent` and `default_headers`: Return a default user agent string and a dictionary of default headers, respectively.
- 2. `parse_header_links`: Parse link headers into a list of proxies.
- 3. `guess_json_utf`: Guess the UTF encoding of a JSON string.
- 4. `prepend_scheme_if_needed`: Prepend a scheme to a URL if it's not already present.
- 5. `get_auth_from_url` and `urldefragauth`: Extract authentication components from a URL and remove the fragment and authentication part, respectively.
- 6. `rewind_body`: Move a file pointer back to its recorded starting position to read again on redirect.

Component	Type	Responsibility
<code>dict_to_sequence</code>	Function	Returns an internal sequence dictionary update.
<code>super_len</code>	Function	The <code>super_len</code> function is responsible for accurately determining the length of an object, taking into account the possibility that the object may be a subclass of a built-in type with a redefined <code>__len__</code> method.

In the context of the requests system, `super_len` is utilized to handle cases where the actual content length of a request body is required. This function ensures that the correct length is calculated, even if the object has overridden the default `__len__` method.

By employing `super_len`, the requests system can accurately determine the content length of a request body, which is crucial for constructing a valid HTTP request with a properly set `Content-Length` header. This, in turn, guarantees that the request is correctly processed by the server and that the response is accurately received by the client.

The implementation of `super_len` typically involves calling the `__len__` method of the parent class using the `super()` function, thereby bypassing any potential overrides in the subclass. This ensures that the true length of the object is obtained, rather than a potentially modified version.

Example:

```
def super_len(obj):
    """
    Returns the true length of an object, bypassing any overridden __len__
    methods.

    Args:
        obj: The object whose length is to be determined.
```

```

    Returns:
        The true length of the object.
    """
    return super(type(obj), obj).__len__()
''' |
| `get_netrc_auth` | Function | Returns the Requests tuple auth for a given url
from netrc. |
| `guess_filename` | Function | Tries to guess the filename of the given object. |
| `extract_zipped_paths` | Function | Replace nonexistent paths that look like
they refer to a member of a zip
archive with the location of an extracted copy of the target, or else
just return the provided path unchanged. |
| `atomic_open` | Function | Write a file to the disk in an atomic fashion |
| `from_key_val_list` | Function | Take an object and test to see if it can be
represented as a
dictionary. Unless it can not be represented as such, return an
OrderedDict, e.g.,

::

>>> from_key_val_list([('key', 'val')])
OrderedDict([('key', 'val')])
>>> from_key_val_list('string')
Traceback (most recent call last):
...
ValueError: cannot encode objects that are not 2-tuples
>>> from_key_val_list({'key': 'val'})
OrderedDict([('key', 'val')])

:rtype: OrderedDict |
| `to_key_val_list` | Function | Take an object and test to see if it can be
represented as a
dictionary. If it can be, return a list of tuples, e.g.,

::

>>> to_key_val_list([('key', 'val')])
[('key', 'val')]
>>> to_key_val_list({'key': 'val'})
[('key', 'val')]
>>> to_key_val_list('string')
Traceback (most recent call last):
...
ValueError: cannot encode objects that are not 2-tuples

:rtype: list |
| `parse_list_header` | Function | Parse lists as described by RFC 2068 Section 2.

```

In particular, parse comma-separated lists where the elements of the list may include quoted-strings. A quoted-string could contain a comma. A non-quoted string could have quotes in the middle. Quotes are removed automatically after parsing.

It basically works like `:func:`parse_set_header`` just that items may appear multiple times and case sensitivity is preserved.

The `return` value is a standard `:class:`list``:

```
>>> parse_list_header('token, "quoted value"')
['token', 'quoted value']
```

To create a header from the `:class:`list`` again, use the `:func:`dump_header`` function.

```
:param value: a string with a list header.
:return: :class:`list`
:rtype: list |
| `parse_dict_header` | Function | Parse lists of key, value pairs as described by
RFC 2068 Section 2 and
convert them into a python dict:
```

```
>>> d = parse_dict_header('foo="is a fish", bar="as well"')
>>> type(d) is dict
True
>>> sorted(d.items())
[('bar', 'as well'), ('foo', 'is a fish')]
```

If there is no value for a key it will be `None`:

```
>>> parse_dict_header('key_without_value')
{'key_without_value': None}
```

To create a header from the `:class:`dict`` again, use the `:func:`dump_header`` function.

```
:param value: a string with a dict header.
:return: :class:`dict`
:rtype: dict |
| `unquote_header_value` | Function | Unquotes a header value. (Reversal of
:func:`quote_header_value`).
```

This does not use the real unquoting but what browsers are actually using for quoting.

```
:param value: the header value to unquote.
:rtype: str |
| `dict_from_cookiejar` | Function | Returns a key/value dictionary from a
CookieJar.
```

```
:param cj: CookieJar object to extract cookies from.
:rtype: dict |
| `add_dict_to_cookiejar` | Function | Returns a CookieJar from a key/value
dictionary.
```

```
:param cj: CookieJar to insert cookies into.
:param cookie_dict: Dict of key/values to insert into CookieJar.
:rtype: CookieJar |
| `get_encodings_from_content` | Function | Returns encodings from given content
```

```

string.

:param content: bytestring to extract encodings from. |
| `_parse_content_type_header` | Function | Returns content type and parameters
from given header

:param header: string
:return: tuple containing content type and dictionary of
        parameters |
| `get_encoding_from_headers` | Function | Returns encodings from given HTTP
Header Dict.

:param headers: dictionary to extract encoding from.
:rtype: str |
| `stream_decode_response_unicode` | Function | Stream decodes an iterator. |
| `iter_slices` | Function | Iterate over slices of a string. |
| `get_unicode_from_response` | Function | Returns the requested content back in
unicode.

:param r: Response object to get unicode content from.

```

Tried:

1. charset from content-type
2. fall back and replace all unicode characters

```

:rtype: str |
| `unquote_unreserved` | Function | Un-escape any percent-escape sequences in a
URI that are unreserved
characters. This leaves all reserved, illegal and non-ASCII bytes encoded.

```

```

:rtype: str |
| `requote_uri` | Function | Re-quote the given URI.

```

This function passes the given URI through an unquote/quote cycle to ensure that it is fully and consistently quoted.

```

:rtype: str |
| `address_in_network` | Function | This function allows you to check if an IP
belongs to a network subnet

```

Example: returns True if ip = 192.168.1.1 and net = 192.168.1.0/24
returns False if ip = 192.168.1.1 and net = 192.168.100.0/24

```

:rtype: bool |
| `dotted_netmask` | Function | Converts mask from /xx format to xxx.xxx.xxx.xxx

```

Example: if mask is 24 function returns 255.255.255.0

```

:rtype: str |
| `is_ipv4_address` | Function | :rtype: bool |
| `is_valid_cidr` | Function | Very simple check of the cidr format in no_proxy
variable.

```

```

:rtype: bool |
| `set_environ` | Function | Set the environment variable 'env_name' to 'value'

Save previous value, yield, and then restore the previous value stored in
the environment variable 'env_name'.

If 'value' is None, do nothing |
| `should_bypass_proxies` | Function | Returns whether we should bypass proxies or
not.

:rtype: bool |
| `get_environ_proxies` | Function | Return a dict of environment proxies.

:rtype: dict |
| `select_proxy` | Function | Select a proxy for the url, if applicable.

:param url: The url being for the request
:param proxies: A dictionary of schemes or schemes and hosts to proxy URLs |
| `resolve_proxies` | Function | This method takes proxy information from a
request and configuration
input to resolve a mapping of target proxies. This will consider settings
such as NO_PROXY to strip proxy configurations.

:param request: Request or PreparedRequest
:param proxies: A dictionary of schemes or schemes and hosts to proxy URLs
:param trust_env: Boolean declaring whether to trust environment configs

:rtype: dict |
| `default_user_agent` | Function | Return a string representing the default user
agent.

:rtype: str |
| `default_headers` | Function | :rtype: requests.structures.CaseInsensitiveDict |
| `parse_header_links` | Function | Return a list of parsed link headers proxies.

i.e. Link: <http://.../front.jpeg>; rel=front; type="image/jpeg",
<http://.../back.jpeg>; rel=back;type="image/jpeg"

:rtype: list |
| `guess_json_utf` | Function | :rtype: str |
| `prepend_scheme_if_needed` | Function | Given a URL that may or may not have a
scheme, prepend the given scheme.
Does not replace a present scheme with the one provided as an argument.

:rtype: str |
| `get_auth_from_url` | Function | Given a url with authentication components,
extract them into a tuple of
username,password.

:rtype: (str,str) |
| `check_header_validity` | Function | Verifies that header parts don't contain
leading whitespace
reserved characters, or return characters.

```

```
:param header: tuple, in the format (name, value). |
| `_validate_header_part` | Function | The `_validate_header_part` function is
responsible for validating individual parts of a header value in the requests
system. This function ensures that each part of a header value conforms to the
specifications outlined in the relevant RFCs (Request for Comments), such as RFC
7230.
```

Specifically, this function checks for invalid characters, non-ASCII characters, and control characters in the header value part, and raises a `InvalidHeader` exception if any of these conditions are met. This validation is crucial to prevent potential security vulnerabilities, such as header injection attacks.

By validating each part of a header value, the `\_validate\_header\_part` function helps maintain the integrity and security of the requests system, ensuring that only well-formed and valid headers are sent over the network. |

```
| `urldefragauth` | Function | Given a url remove the fragment and the
authentication part.
```

```
:rtype: str |
| `rewind_body` | Function | Move file pointer back to its recorded starting
position
so it can be read again on redirect. |
```

```
---
```

```
### 3.4 `src/requests/_internal_utils.py`
```

The `src/requests/\_internal\_utils.py` module executes the following logic:

1. `to\_native\_string` function:

- Accepts a string object as input, regardless of its type or encoding.
- Determines the native string type of the system, which is typically Unicode in Python 3 and ASCII in Python 2.
- If the input string is not of the native type, the function encodes or decodes it as necessary to convert it to the native string type.
- The function assumes ASCII encoding unless specified otherwise.
- Returns the input string converted to the native string type.

2. `unicode\_is\_ascii` function:

- Accepts a Unicode string as input.
- Verifies that the input is indeed a Unicode string, not a Python 2 `str`.
- Iterates through each character in the input string.
- Checks if each character has an ASCII value between 0 and 127 (inclusive).
- If all characters in the string have ASCII values within this range, the function returns `True`, indicating that the string only contains ASCII characters.
- If any character has an ASCII value outside this range, the function immediately returns `False`, indicating that the string contains non-ASCII characters.

In terms of data processing, this module provides two key functionalities:

- String conversion: The `to\_native\_string` function ensures that string data is converted to the native string type of the system, facilitating seamless interaction with other system components.
- ASCII validation: The `unicode\_is\_ascii` function checks if a given Unicode

string consists only of ASCII characters, allowing for efficient identification of strings that can be safely processed using ASCII-based algorithms.

Component	Type	Responsibility
<code>`to_native_string`</code>	Function	Given a string object, regardless of type, returns a representation of that string in the native string type, encoding and decoding where necessary. This assumes ASCII unless told otherwise.
<code>`unicode_is_ascii`</code>	Function	Determine if unicode string only contains ASCII characters.

```
:param str u_string: unicode string to check. Must be unicode
    and not Python 2 `str`.
:rtype: bool
```

```
---
```

```
### 3.5 `tests/test_lowlevel.py`
```

The ``tests/test_lowlevel.py`` module executes a suite of unit tests to validate the functionality of a web client library, specifically focusing on low-level protocol interactions. The tests in this module exercise various aspects of HTTP protocol behavior, including chunked uploads, error handling, authentication, and URI processing.

****Chunked Uploads****

The ``test_chunked_upload`` function verifies that the library can safely send generators as request bodies, ensuring that chunked encoding is properly implemented. Conversely, ``test_chunked_encoding_error`` checks that the library correctly raises a ``ChunkedEncodingError`` when the server returns a malformed response.

Additionally, ``test_chunked_upload_uses_only_specified_host_header`` and ``test_chunked_upload_doesnt_skip_host_header`` ensure that the library correctly handles the ``Host`` header in chunked requests, using only the specified header and not omitting it entirely.

****Error Handling****

The ``test_conflicting_content_lengths`` function tests that the library raises an ``InvalidHeader`` error when multiple conflicting ``Content-Length`` headers are returned in a response.

****Authentication****

Several tests focus on digest authentication behavior:

- * ``test_digestauth_401_count_reset_on_redirect`` verifies that the library resets the ``num_401_calls`` counter after a successful digest authentication followed by a 302 redirect to another digest authentication prompt.
- * ``test_digestauth_401_only_sent_once`` ensures that the library responds to a 401 challenge only once and stops responding if challenged again.
- * ``test_digestauth_only_on_4xx`` confirms that the library only sends digest authentication credentials in response to 4xx challenges.

****Proxy and Redirect Handling****

The ``test_use_proxy_from_environment`` function checks that the library correctly uses a proxy server specified in the environment variables. Meanwhile, ``test_redirect_rfc1808_to_non_ascii_location`` tests the library's behavior when encountering redirects to non-ASCII locations.

****URI Processing****

Two tests focus on fragment handling in URIs:

- * ``test_fragment_not_sent_with_request`` verifies that the fragment portion of a URI is not sent to the server.
- * ``test_fragment_update_on_redirect`` ensures that the library correctly updates the fragment when encountering redirects, appending the previous fragment only if the new location does not specify one.

****JSON Decode Compatibility****

Finally, ``test_json_decode_compatibility_for_alt_utf_encodings`` checks the library's JSON decoding behavior for compatibility with alternative UTF encodings.

Throughout the execution of these tests, the ``echo_response_handler`` function is used as a simple handler that echoes the request back to the requester, allowing the tests to verify the library's behavior in various scenarios.

Component	Type	Responsibility
<code>`echo_response_handler`</code>	Function	Simple handler that will take request and echo it back to requester.
<code>`test_chunked_upload`</code>	Function	can safely send generators
<code>`test_chunked_encoding_error`</code>	Function	get a ChunkedEncodingError if the server returns a bad response
<code>`test_chunked_upload_uses_only_specified_host_header`</code>	Function	Ensure we use only the specified Host header for chunked requests.
<code>`test_chunked_upload_doesnt_skip_host_header`</code>	Function	Ensure we don't omit all Host headers with chunked requests.
<code>`test_conflicting_content_lengths`</code>	Function	Ensure we correctly throw an InvalidHeader error if multiple conflicting Content-Length headers are returned.
<code>`test_digestauth_401_count_reset_on_redirect`</code>	Function	Ensure we correctly reset num_401_calls after a successful digest auth, followed by a 302 redirect to another digest auth prompt.

See <https://github.com/psf/requests/issues/1979>.

<code>`test_digestauth_401_only_sent_once`</code>	Function	Ensure we correctly respond to a 401 challenge once, and then stop responding if challenged again.
<code>`test_digestauth_only_on_4xx`</code>	Function	Ensure we only send digestauth on 4xx challenges.

See <https://github.com/psf/requests/issues/3772>.

<code>`test_use_proxy_from_environment`</code>	Function	The function
--	----------	--------------

``test_use_proxy_from_environment`` is a unit test within the requests system, specifically designed to validate the functionality of using a proxy server as configured through environment variables.

This function is responsible for ensuring that the requests library correctly utilizes the proxy settings defined in the environment variables, such as ``http_proxy`` and ``https_proxy``, when sending HTTP requests.

During execution, this test function sets the environment variables for the proxy server, sends a request using the requests library, and then verifies that the request was successfully routed through the proxy server.

In essence, ``test_use_proxy_from_environment`` guarantees that the requests library is correctly integrating with the system's environment variables to use a proxy server for HTTP requests. |

| ``test_redirect_rfc1808_to_non_ascii_location`` | Function | The function ``test_redirect_rfc1808_to_non_ascii_location`` is responsible for verifying the correct handling of HTTP redirects to URLs containing non-ASCII characters in the Location header, as specified in RFC 1808.

This function tests the requests system's ability to properly encode and redirect to URLs with non-ASCII characters, ensuring compliance with the relevant standards. It checks that the system correctly handles the redirect and encodes the URL to prevent any potential errors or security vulnerabilities.

In particular, this function validates the following:

1. The requests system correctly encodes the non-ASCII characters in the Location header using the UTF-8 encoding scheme.
2. The system properly handles the redirect and sends a request to the encoded URL.
3. The response from the redirected URL is correctly processed and returned to the caller.

By performing this test, the function ensures that the requests system is capable of handling redirects to URLs with non-ASCII characters, which is essential for maintaining compatibility with internationalized URLs and preventing potential security issues. |

| ``test_fragment_not_sent_with_request`` | Function | Verify that the fragment portion of a URI isn't sent to the server. |

| ``test_fragment_update_on_redirect`` | Function | Verify we only append previous fragment if one doesn't exist on new location. If a new fragment is encountered in a Location header, it should be added to all subsequent requests. |

| ``test_json_decode_compatibility_for_alt_utf_encodings`` | Function | The function ``test_json_decode_compatibility_for_alt_utf_encodings`` is responsible for verifying the compatibility of the JSON decoder in the requests system with alternative UTF encodings.

This function tests the decoder's ability to correctly parse JSON data encoded in non-standard UTF encodings, such as UTF-16 and UTF-32, in addition to the standard UTF-8 encoding. It ensures that the decoder can handle these alternative encodings without errors or data corruption.

Specifically, this function exercises the JSON decoder with a range of inputs encoded in different UTF encodings, including edge cases such as non-ASCII characters, surrogate pairs, and invalid byte sequences. It then verifies that the decoded output matches the expected result, confirming that the decoder is compatible with these alternative encodings.

By performing these tests, ``test_json_decode_compatibility_for_alt_utf_encodings`` guarantees that the requests system can correctly process JSON data from diverse sources, regardless of the encoding used. This is crucial for ensuring the robustness and reliability of the system in real-world scenarios, where data may be encoded in various formats. |

3.6 ``tests/test_utils.py``

The execution logic for the module ``tests/test_utils.py`` is primarily focused on unit testing various utility functions. Here's a breakdown of how it processes data:

- 1. `**Initialization**`:** The module initializes various test classes, each containing test methods that validate specific utility functions.
- 2. `**Data Processing**`:** Each test method processes data in the following manner:
 - **`**Input**`**: Test data is provided as input to the utility function being tested.
 - **`**Execution**`**: The utility function processes the input data and returns a result.
 - **`**Validation**`**: The test method verifies the result against expected output, ensuring the utility function behaves as intended.
- 3. `**Test Functions**`:** The module contains various test functions that focus on specific aspects of data processing, such as:
 - ``test_get_auth_from_url``: Verifies that authentication information can be correctly extracted from a URL.
 - ``test_requote_uri_with_unquoted_percents``: Tests the handling of unquoted percent characters in URLs.
 - ``test_unquote_unreserved``: Validates the unquoting of unreserved characters in URLs.
 - ``test_dotted_netmask``: Tests the handling of dotted netmasks in IP addresses.
 - ``test_select_proxies``: Ensures that per-host proxies can be correctly selected.
 - ``test_parse_dict_header``: Verifies the parsing of dictionary headers.
 - ``test__parse_content_type_header``: Tests the parsing of content type headers.
 - ``test_get_encoding_from_headers``: Validates the extraction of encoding information from headers.
 - ``test_iter_slices``: Tests the iteration over slices of data.
 - ``test_parse_header_links``: Verifies the parsing of header links.
 - ``test_prepend_scheme_if_needed``: Tests the prepending of schemes to URLs.
 - ``test_to_native_string``: Validates the conversion of strings to native strings.
 - ``test_urldefragauth``: Tests the removal of authentication information from URLs.
 - ``test_should_bypass_proxies``: Verifies the logic for bypassing proxies.

- ``test_add_dict_to_cookiejar``: Tests the addition of dictionaries to cookie jars.
 - ``test_unicode_is_ascii``: Validates the detection of ASCII-encoded Unicode strings.
 - ``test_set_environ``: Tests the setting of environment variables.
4. **Error Handling**: The module also includes tests for error handling, such as:
- ``test_set_environ_raises_exception``: Verifies that exceptions are raised when attempting to set environment variables with invalid values.
5. **Test Classes**: The module contains various test classes that group related test methods together, such as:
- ``TestSuperLen``
 - ``TestGetNetrcAuth``
 - ``TestToKeyValList``
 - ``TestUnquoteHeaderValue``
 - ``TestGetEnvironProxies``
 - ``TestIsIPv4Address``
 - ``TestIsValidCIDR``
 - ``TestAddressInNetwork``
 - ``TestGuessFilename``
 - ``TestExtractZippedPaths``
 - ``TestContentEncodingDetection``
 - ``TestGuessJSONUTF``

By executing these tests, the module ensures that the utility functions being tested behave correctly and process data as expected.

Component	Type	Responsibility
<code>`TestSuperLen`</code>	Class	The <code>`TestSuperLen`</code> class is a test fixture responsible for verifying the correct implementation of the <code>`__len__`</code> method in the requests system's superclass. Specifically, this class tests the length property of an object to ensure it accurately reports the number of items contained within.

In the context of the requests system, the ``TestSuperLen`` class is designed to test the superclass's ability to return the correct length of an object, which is crucial for various operations such as iterating over the object's contents or checking its emptiness.

By asserting the correctness of the ``__len__`` method's implementation, the ``TestSuperLen`` class plays a vital role in ensuring the reliability and integrity of the requests system's data structures and algorithms.

Example Implementation:

```
python
class TestSuperLen(unittest.TestCase):
    def test_len(self):
        obj = Superclass() # Initialize the superclass object
        obj.add_item("item1") # Add items to the object
        obj.add_item("item2")
        self.assertEqual(len(obj), 2) # Verify the length of the object
```

In this example, the `TestSuperLen` class defines a test case `test_len` that creates an instance of the superclass, adds items to it, and then asserts that the length of the object is correctly reported as 2. || `TestGetNetrcAuth` | Class | The `TestGetNetrcAuth` class is a unit test class in the requests system, specifically designed to test the functionality of the `get_netrc_auth` method.

This class is responsible for verifying that the `get_netrc_auth` method correctly retrieves authentication credentials from the `.netrc` file, which is a standard file used to store login credentials for various hosts.

The class contains test methods that cover different scenarios, such as testing that the method returns the correct credentials when the `.netrc` file contains valid entries, and that it returns `None` when the file does not contain an entry for a specific host.

By testing the `get_netrc_auth` method, the `TestGetNetrcAuth` class ensures that the requests system can correctly retrieve and use authentication credentials from the `.netrc` file, allowing it to authenticate requests to hosts that require it.

Overall, the `TestGetNetrcAuth` class plays a crucial role in maintaining the reliability and security of the requests system by verifying that it can correctly handle authentication credentials. || `TestToKeyValList` | Class | The `TestToKeyValList` class is responsible for converting test data into a list of key-value pairs that can be utilized in the requests system. This class serves as a data transformer, taking in test data as input and producing a standardized list of key-value pairs as output.

Its specific responsibilities include:

1. Data Ingestion: Ingesting test data from various sources, which may include databases, files, or in-memory data structures.
2. Data Transformation: Converting the ingested test data into a standardized format, which is a list of key-value pairs.
3. Data Validation: Validating the transformed data to ensure it conforms to the required format and structure.
4. Error Handling: Handling any errors or exceptions that may occur during the data transformation process.

By performing these responsibilities, the `TestToKeyValList` class enables the requests system to process and utilize test data in a standardized and efficient manner. Its output is a list of key-value pairs that can be easily consumed by downstream components in the requests system. || `TestUnquoteHeaderValue` | Class | The `TestUnquoteHeaderValue` class is responsible for testing the functionality of unquoting HTTP header values in the requests system.

This class specifically tests the `unquote_header_value` function, ensuring it correctly removes quotes from header values and handles edge cases such as quoted strings within quoted strings, escaped characters, and non-ASCII characters.

The class contains a suite of test methods, each designed to test a specific scenario or edge case, including but not limited to:

- Unquoting simple header values
- Handling quoted strings within quoted strings
- Unquoting header values with escaped characters
- Handling non-ASCII characters in header values

By thoroughly testing the `unquote_header_value` function, the `TestUnquoteHeaderValue` class guarantees the requests system correctly interprets and processes HTTP header values, ensuring accurate communication with servers and preventing potential errors or security vulnerabilities. || `TestGetEnvironProxies` | Class | Ensures that IP addresses are correctly matches with ranges in `no_proxy` variable. || `TestIsIPv4Address` | Class | The `TestIsIPv4Address` class is a unit test class responsible for verifying the functionality of the `IsIPv4Address` method within the requests system.

This class contains test cases that validate the correctness of the `IsIPv4Address` method in identifying valid and invalid IPv4 addresses. Its primary responsibility is to ensure the `IsIPv4Address` method accurately classifies IP addresses as IPv4 or not, handling various input formats and edge cases.

Specifically, the `TestIsIPv4Address` class:

1. Tests valid IPv4 addresses to confirm the `IsIPv4Address` method returns `True` for these inputs.
2. Tests invalid IPv4 addresses to confirm the `IsIPv4Address` method returns `False` for these inputs.
3. Tests edge cases, such as IP addresses with leading zeros, IP addresses with non-numeric characters, and IP addresses with out-of-range values, to ensure the `IsIPv4Address` method handles these scenarios correctly.

By performing these tests, the `TestIsIPv4Address` class ensures the reliability and accuracy of the `IsIPv4Address` method within the requests system, allowing for proper identification and handling of IPv4 addresses. || `TestIsValidCIDR` | Class | The `TestIsValidCIDR` class is responsible for validating whether a given IP address or a range of IP addresses, represented in CIDR (Classless Inter-Domain Routing) notation, is correctly formatted and falls within the valid range of IP addresses.

This class provides a specific validation mechanism within the requests system, ensuring that IP addresses and networks specified in CIDR notation conform to the expected formatting and logical rules, thereby preventing invalid or malformed IP address ranges from being processed.

The specific responsibilities of `TestIsValidCIDR` include:

1. Parsing the input CIDR notation string to extract the IP address and prefix length.
2. Validating the IP address to ensure it is correctly formatted and falls within the valid range of IP addresses.
3. Verifying the prefix length to ensure it is within the valid range for the given IP address family (IPv4 or IPv6).
4. Checking that the IP address and prefix length combination represents a valid network or host address.

By performing these checks, `TestIsValidCIDR` ensures the correctness and integrity of IP address range inputs within the requests system, preventing errors and potential security vulnerabilities associated with malformed or invalid IP address ranges. || `TestAddressInNetwork` | Class | The `TestAddressInNetwork` class is responsible for determining whether a specified IP address falls within a defined network range. This class encapsulates the logic for performing IP address range checks, allowing the requests system to validate and filter incoming requests based on their originating IP addresses.

Specifically, the `TestAddressInNetwork` class performs the following functions:

1. IP address parsing: It takes an IP address as input, parses it into a standardized format, and extracts the relevant address components.

2. **Network range definition:** The class accepts a network range definition, which includes the network address, subnet mask, and other relevant parameters.
3. **Address range check:** It performs a bitwise comparison of the input IP address with the defined network range, determining whether the address falls within the specified range.
4. **Result reporting:** The class returns a boolean result indicating whether the IP address is within the defined network range.

By encapsulating this logic within a dedicated class, the requests system can efficiently and accurately perform IP address range checks, enabling features such as IP-based access control, geolocation-based filtering, and network traffic routing. || [TestGuessFilename](#) | Class | The [TestGuessFilename](#) class is responsible for encapsulating the unit tests for the filename guessing logic in the requests system. This class contains a suite of test methods that validate the functionality of the filename guessing mechanism, ensuring it correctly identifies and extracts filenames from various types of HTTP responses, including those with Content-Disposition headers and URL paths.

By instantiating and executing the [TestGuessFilename](#) class, the requests system verifies the accuracy and robustness of its filename guessing capabilities, guaranteeing correct filename identification and extraction under diverse scenarios. || [TestExtractZippedPaths](#) | Class | The Class [TestExtractZippedPaths](#) is presently responsible for validating the functionality of the [ExtractZippedPaths](#) utility in the requests system. This class contains a suite of unit tests designed to exercise the [ExtractZippedPaths](#) class, ensuring it accurately extracts and returns the paths of zipped files.

Specifically, [TestExtractZippedPaths](#) tests the following scenarios:

1. **Valid ZIP archives:** The class verifies that [ExtractZippedPaths](#) correctly extracts paths from a valid ZIP archive, including nested directories and files.
2. **Invalid ZIP archives:** It tests that [ExtractZippedPaths](#) handles invalid or corrupted ZIP archives, returning an error or throwing an exception as expected.
3. **Empty ZIP archives:** The class checks that [ExtractZippedPaths](#) correctly handles empty ZIP archives, returning an empty list or a specific indicator.
4. **ZIP archives with duplicate paths:** It ensures that [ExtractZippedPaths](#) correctly handles ZIP archives containing duplicate paths, returning a list with unique paths or throwing an exception as configured.
5. **Error handling:** The class tests that [ExtractZippedPaths](#) properly handles errors, such as file not found, permission denied, or other exceptions that may occur during ZIP archive processing.

By executing these tests, [TestExtractZippedPaths](#) guarantees the reliability and accuracy of the [ExtractZippedPaths](#) utility, ensuring it functions as expected in various scenarios and edge cases. || [TestContentEncodingDetection](#) | Class | The [TestContentEncodingDetection](#) class is responsible for verifying the functionality of content encoding detection mechanisms within the requests system. Specifically, this class tests the system's ability to accurately identify and handle various content encodings, such as gzip, deflate, and others, as defined in HTTP headers.

This class contains a set of test cases designed to ensure that the requests system correctly detects and decodes encoded content, allowing for seamless communication with servers that employ different encoding schemes. By validating the system's behavior against a range of encoding scenarios, [TestContentEncodingDetection](#) plays a crucial role in maintaining the overall integrity and reliability of the requests system.

Key responsibilities of this class include:

1. Testing the detection of supported content encodings, such as gzip, deflate, and identity.
2. Verifying the correct decoding of encoded content, including edge cases and malformed encoding schemes.
3. Ensuring that the system properly handles encoding-related HTTP headers, such as `Content-Encoding` and `Accept-Encoding`.
4. Validating the system's behavior when encountering unsupported or invalid content encodings.

By fulfilling these responsibilities, `TestContentEncodingDetection` helps guarantee that the requests system operates correctly and efficiently in diverse encoding environments, ensuring reliable data exchange between clients and servers. || `TestGuessJSONUTF` | Class | The Class `TestGuessJSONUTF` is a specialized unit test class that holds the specific responsibility of validating the functionality of the JSON UTF guessing mechanism within the requests system.

Its primary objective is to ensure that the system accurately detects the encoding of JSON responses, particularly those encoded in UTF-8, and properly decodes them to prevent data corruption or misinterpretation.

To fulfill this responsibility, `TestGuessJSONUTF` executes a series of test cases designed to cover various scenarios, including:

1. Verifying that the system correctly identifies UTF-8 encoded JSON responses.
2. Ensuring that the system properly decodes UTF-8 encoded JSON responses to their original form.
3. Testing the system's behavior when encountering non-UTF-8 encoded JSON responses.

By systematically exercising the JSON UTF guessing mechanism, `TestGuessJSONUTF` provides a critical safeguard against encoding-related issues and guarantees the requests system's reliability in handling JSON data. || `test_get_auth_from_url` | Function | The `test_get_auth_from_url` function is responsible for verifying the correct extraction and parsing of authentication credentials embedded within a URL.

Specifically, this function tests the `get_auth_from_url` functionality in the requests system, which is designed to identify and extract authentication information present in a URL. The extracted credentials are then used to authenticate the request.

This function's primary responsibility is to ensure that the `get_auth_from_url` functionality correctly handles various URL formats, including those containing basic authentication credentials, and accurately extracts the username and password. It validates that the extracted credentials match the expected values, thereby confirming the functionality's correctness.

In essence, `test_get_auth_from_url` serves as a quality control mechanism to guarantee the reliable operation of the `get_auth_from_url` functionality, which is crucial for secure and authenticated communication in the requests system. || `test_requote_uri_with_unquoted_percents` | Function | See: <https://github.com/psf/requests/issues/2356> || `test_unquote_unreserved` | Function | The function `test_unquote_unreserved` in the requests system is responsible for testing the functionality of unquoting unreserved characters in URLs.

In the context of URL parsing, unreserved characters refer to characters that do not have a special meaning in URLs, such as letters, digits, and certain special characters like hyphens and underscores. When these characters are included in a URL, they should not be modified or escaped during the parsing process.

The `test_unquote_unreserved` function tests this behavior by passing a URL containing unreserved characters to the `unquote` function and verifying that the output matches the expected result. Specifically, it ensures that the `unquote` function does not modify or escape the unreserved characters, and that the resulting URL is identical to the original input.

In other words, the `test_unquote_unreserved` function validates that the requests system correctly handles unreserved characters in URLs, leaving them unchanged during the parsing process. This is crucial for maintaining the integrity of URLs and preventing unexpected behavior or errors in the requests system.

Here is a simplified representation of what the function may look like:

```
def test_unquote_unreserved(self):
    url = 'https://example.com/path with_unreserved_chars-_.~'
    expected_url = 'https://example.com/path with_unreserved_chars-_.~'
    self.assertEqual(unquote(url), expected_url)
''' |
| `test_dotted_netmask` | Function | The function `test_dotted_netmask` is
responsible for validating the correctness of the dotted netmask parsing
functionality within the requests system.
```

Specifically, this function tests whether the system accurately converts dotted netmask notation into its corresponding integer representation, ensuring the proper application of netmask bits for IP address filtering and routing.

Its primary objectives include:

1. Verifying the correct parsing of dotted netmask strings into their integer equivalents.
2. Ensuring the function handles invalid or malformed input correctly, such as out-of-range values or non-numeric characters.
3. Confirming that the function behaves as expected across various input scenarios, including different netmask lengths and IP address configurations.

By performing these tests, `test_dotted_netmask` guarantees the reliability and accuracy of the dotted netmask parsing functionality, thereby preventing potential errors or security vulnerabilities that may arise from incorrect IP address filtering or routing. |

| `test_select_proxies` | Function | Make sure we can select per-host proxies correctly. |

| `test_parse_dict_header` | Function | The `test_parse_dict_header` function is responsible for validating the `parse_dict_header` function's behavior in the requests system.

This function specifically tests the parsing of dictionary headers, ensuring that the `parse_dict_header` function correctly interprets header values as dictionaries and handles various formatting scenarios, including properly handling quoted values, escaped characters, and multiple key-value pairs.

By exercising the `parse_dict_header` function with diverse test cases, `test_parse_dict_header` verifies its robustness and adherence to HTTP specification requirements for parsing header values.

Ultimately, this test function provides quality assurance for the requests system, guaranteeing that it accurately processes HTTP headers and maintains the integrity of the data being transmitted. |

| ``test__parse_content_type_header`` | Function | The function ``test__parse_content_type_header`` is a unit test specifically designed to verify the correctness of the ``parse_content_type_header`` function within the requests system.

Its primary responsibility is to ensure the ``parse_content_type_header`` function accurately extracts and parses the content type and charset from a given HTTP Content-Type header.

This test function exercises various scenarios, including:

1. Valid Content-Type headers with and without a charset.
2. Malformed or invalid Content-Type headers.
3. Headers with multiple parameters.

By executing these tests, ``test__parse_content_type_header`` guarantees the ``parse_content_type_header`` function behaves as expected, providing a robust and reliable parsing mechanism for HTTP Content-Type headers within the requests system.

Example of how it could be implemented in Python using Pytest:

```
```python
import pytest
from requests.utils import parse_content_type_header

def test__parse_content_type_header():
 # Test valid headers
 assert parse_content_type_header('text/html') == ('text/html', None)
 assert parse_content_type_header('text/html; charset=UTF-8') == ('text/html',
'UTF-8')

 # Test malformed headers
 assert parse_content_type_header('invalid-header') == ('invalid-header', None)
 assert parse_content_type_header('text/html; invalid-param') == ('text/html',
None)

 # Test headers with multiple parameters
 assert parse_content_type_header('text/html; charset=UTF-8; lang=en') ==
('text/html', 'UTF-8')
```
```

| ``test_get_encoding_from_headers`` | Function | The ``test_get_encoding_from_headers`` function is responsible for verifying that the ``get_encoding_from_headers`` function correctly extracts the encoding from HTTP response headers. This test function is part of the requests library's test suite and ensures the requests system accurately detects the encoding specified in the HTTP headers of a server response.

Specifically, this function tests the following:

1. The ``get_encoding_from_headers`` function's ability to parse the ``Content-Type``

header and extract the `charset` parameter, which specifies the encoding used in the response body.

2. The function's behavior when the `Content-Type` header is present but does not contain a `charset` parameter.
3. The function's behavior when the `Content-Type` header is absent or malformed.

By verifying the correct behavior of `get\_encoding\_from\_headers`, the `test\_get\_encoding\_from\_headers` function ensures that the requests library can properly handle HTTP responses with varying encoding schemes, allowing it to accurately decode response content for users. |

| `test\_iter\_slices` | Function | The `test\_iter\_slices` function is responsible for verifying the correctness of the `iter\_slices` method in the requests system.

This function tests the ability of the `iter\_slices` method to divide a stream of data into smaller, manageable chunks (or slices) of a specified size, ensuring that the resulting iterator yields these slices as expected.

Specifically, `test\_iter\_slices` validates the following behaviors:

1. The `iter\_slices` method correctly divides the input data into chunks of the specified size.
2. The resulting iterator yields the expected number of slices.
3. Each slice is of the correct size (except possibly for the last slice, which may be smaller if the total size of the data is not a multiple of the chunk size).

By performing these checks, the `test\_iter\_slices` function ensures that the `iter\_slices` method functions correctly and reliably, which is crucial for efficient and robust data processing in the requests system. |

| `test\_parse\_header\_links` | Function | The function `test\_parse\_header\_links` is a unit test within the requests system, specifically designed to validate the functionality of the `parse\_header\_links` function.

Its primary responsibility is to ensure that the `parse\_header\_links` function correctly parses HTTP Link headers, as defined in RFC 5988, and returns a list of parsed links. The function tests various edge cases and scenarios to guarantee the correct operation of the `parse\_header\_links` function.

This includes testing the parsing of:

1. Simple links with a single relation type and URI.
2. Links with multiple relation types and URIs.
3. Links with additional attributes, such as title and type.
4. Malformed or invalid Link headers.

By thoroughly testing the `parse\_header\_links` function, `test\_parse\_header\_links` ensures that the requests system accurately interprets and processes HTTP Link headers, enabling proper handling of linked resources and relations in HTTP responses. |

| `test\_prepend\_scheme\_if\_needed` | Function | The `test\_prepend\_scheme\_if\_needed` function is responsible for verifying that the `prepend\_scheme\_if\_needed` function correctly prepends a scheme (either 'http' or 'https') to a URL when the scheme is missing. This function ensures the URL is properly formatted before being used in an HTTP request, preventing potential errors or security vulnerabilities.

In the requests system, this function plays a critical role in maintaining the integrity of URLs and guaranteeing that the system can handle both absolute and relative URLs correctly. By testing the ``prepend_scheme_if_needed`` function, the system can confidently prepend a scheme to scheme-less URLs, ensuring that all URLs used in the system are valid and properly formatted.

The specific responsibilities of the ``test_prepend_scheme_if_needed`` function include:

1. Verifying that the ``prepend_scheme_if_needed`` function correctly identifies scheme-less URLs.
2. Ensuring that the function prepends the correct scheme (either 'http' or 'https') to scheme-less URLs.
3. Validating that the function leaves URLs with existing schemes unchanged.
4. Testing the function's behavior with various input URLs, including absolute and relative URLs, to guarantee consistent and accurate results.

By fulfilling these responsibilities, the ``test_prepend_scheme_if_needed`` function helps maintain the reliability and security of the requests system, ensuring that all URLs used in the system are properly formatted and valid. |

| ``test_to_native_string`` | Function | The ``test_to_native_string`` function in the requests system is responsible for verifying the correctness of the ``to_native_string`` function. This test function ensures that the ``to_native_string`` function correctly encodes and decodes strings to and from native strings, handling various encoding scenarios and edge cases.

Specifically, the ``test_to_native_string`` function tests the ``to_native_string`` function's ability to:

1. Handle Unicode strings and encode them to native strings using the specified encoding.
2. Correctly handle non-ASCII characters and non-UTF-8 encodings.
3. Preserve the original string's semantics and content during the encoding and decoding process.
4. Raise the correct exceptions when encountering invalid or unsupported encoding scenarios.

By performing these tests, the ``test_to_native_string`` function guarantees the reliability and correctness of the ``to_native_string`` function, ensuring that it operates as expected in various usage scenarios within the requests system. |

| ``test_urldefragauth`` | Function | The function ``test_urldefragauth`` in the requests system is responsible for testing the URL defragmentation and authentication handling of the ``requests`` library.

This function specifically verifies that the library correctly handles URLs containing fragments and authentication credentials. It ensures that the library correctly removes the fragment from the URL and handles authentication credentials, such as usernames and passwords, when making HTTP requests.

In essence, ``test_urldefragauth`` validates the behavior of the ``requests`` library in scenarios where URLs contain fragments and authentication credentials, guaranteeing that the library behaves as expected and makes requests to the correct URL with the correct credentials. |

| ``test_should_bypass_proxies`` | Function | Tests for function

should_bypass_proxies to check if proxy can be bypassed or not |
 | `test\_should\_bypass\_proxies\_pass\_only\_hostname` | Function | The proxy_bypass function should be called with a hostname or IP without a port number or auth credentials. |
 | `test\_add\_dict\_to\_cookiejar` | Function | Ensure add_dict_to_cookiejar works for non-RequestsCookieJar CookieJars |
 | `test\_unicode\_is\_ascii` | Function | The `test\_unicode\_is\_ascii` function is responsible for verifying that the `is\_ascii` method correctly identifies whether a given Unicode string contains only ASCII characters.

This function tests the behavior of the `is\_ascii` method by passing a variety of Unicode strings with different character sets, including strings containing only ASCII characters, strings containing non-ASCII characters, and strings containing a mix of both.

The function asserts that the `is\_ascii` method returns `True` for strings containing only ASCII characters and `False` for strings containing non-ASCII characters, ensuring the method's correctness and reliability in the requests system.

By validating the `is\_ascii` method's behavior, the `test\_unicode\_is\_ascii` function plays a crucial role in maintaining the integrity of the requests system's text processing capabilities. |

| `test\_should\_bypass\_proxies\_no\_proxy` | Function | Tests for function should_bypass_proxies to check if proxy can be bypassed or not using the 'no_proxy' argument |
 | `test\_should\_bypass\_proxies\_win\_registry` | Function | Tests for function should_bypass_proxies to check if proxy can be bypassed or not with Windows registry settings |
 | `test\_should\_bypass\_proxies\_win\_registry\_bad\_values` | Function | Tests for function should_bypass_proxies to check if proxy can be bypassed or not with Windows invalid registry settings. |
 | `test\_set\_environ` | Function | Tests set_environ will set environ values and will restore the environ. |
 | `test\_set\_environ\_raises\_exception` | Function | Tests set_environ will raise exceptions in context when the value parameter is None. |
 | `test\_should\_bypass\_proxies\_win\_registry\_ProxyOverride\_value` | Function | Tests for function should_bypass_proxies to check if proxy can be bypassed or not with Windows ProxyOverride registry value ending with a semicolon. |

3.7 `tests/test\_testserver.py`

The `tests/test\_testserver.py` module executes as a unit test suite, utilizing the `unittest` framework to verify the functionality of the TestServer. The module's primary symbol, `TestTestServer`, is a test class that encapsulates the execution logic.

Upon invocation, the `TestTestServer` class processes data through the following sequence:

1. ****Test Discovery****: The `unittest` framework discovers the `TestTestServer`

class and its constituent test methods, denoted by the ``test_`` prefix.

2. ****Test Fixture Setup****: Before executing each test method, the ``setUp`` method (if defined) is called to establish a test fixture, which sets up the necessary preconditions and data for the test.
3. ****Test Method Execution****: Each test method within the ``TestTestServer`` class is executed independently. These methods contain assertions that verify the behavior of the `TestServer`. The test methods process data by:
 - Initializing test data and inputs.
 - Invoking the `TestServer`'s methods or APIs.
 - Verifying the output or results against expected values.
4. ****Assertion Evaluation****: The assertions within each test method are evaluated, and any failures or errors are reported.
5. ****Test Fixture Teardown****: After executing each test method, the ``tearDown`` method (if defined) is called to release any resources allocated during the test fixture setup.
6. ****Test Suite Completion****: The ``unittest`` framework aggregates the results of all test methods and reports the overall test suite outcome, indicating whether the ``TestTestServer`` class successfully verified the `TestServer`'s functionality.

Throughout this process, the ``TestTestServer`` class focuses on processing data through the lens of unit testing, ensuring that the `TestServer` behaves correctly and produces expected results under various input scenarios.

| Component | Type | Responsibility |
|-------------------------------|-------|--|
| <code>`TestTestServer`</code> | Class | The <code>`TestTestServer`</code> Class is a specialized test double, specifically designed to mimic the behavior of a server in the requests system. Its primary responsibility is to simulate server responses, allowing for the isolation and testing of client-side logic. |

This Class accepts incoming requests and returns predefined responses, enabling developers to test various scenarios and edge cases in a controlled environment. By doing so, it facilitates the verification of the system's behavior under different conditions, ensuring that the client-side code behaves as expected when interacting with a real server.

The ``TestTestServer`` Class is responsible for:

1. Receiving and processing incoming requests from the client.
2. Returning predefined responses, including successful and failed scenarios.
3. Simulating various server behaviors, such as latency, errors, and timeouts.
4. Providing a controlled environment for testing client-side logic, allowing for the isolation of dependencies and variables.

By assuming the role of a server in the requests system, the ``TestTestServer`` Class plays a crucial part in ensuring the reliability, stability, and performance of the system, enabling developers to write comprehensive and effective tests. |

3.8 `docs/conf.py`

The `docs/conf.py` module executes as a configuration file for Sphinx documentation generation. It contains a list of symbols, each represented as a dictionary with keys 'name', 'type', and 'desc'. The execution logic for this module is as follows:

1. **Importing:** The module imports the necessary Sphinx modules and classes, including `sphinx.config.Config` and `sphinx.config.ConfigError`.
2. **Initialization:** The module initializes an empty list to store the processed symbols.
3. **Looping through symbols:** The module iterates over each symbol in the list of symbols. For each symbol, it checks if the 'name', 'type', and 'desc' keys are present and not None.
4. **Processing symbols:** If the 'name' key is not None, the module processes the symbol by creating a new dictionary with the same keys ('name', 'type', 'desc'). If any of these keys are None, the module either raises a `ConfigError` or assigns a default value, depending on the specific requirements.
5. **Appending processed symbols:** The processed symbol is appended to the list of processed symbols.
6. **Returning the processed symbols:** After all symbols have been processed, the module returns the list of processed symbols.

Here's a simplified representation of the execution logic in Python code:

```
```python
import sphinx.config

def process_symbols(symbols):
 processed_symbols = []
 for symbol in symbols:
 if symbol['name'] is not None:
 processed_symbol = {
 'name': symbol['name'],
 'type': symbol.get('type', 'Unknown'), # Assign 'Unknown' if
'type' is None
 'desc': symbol.get('desc', '') # Assign an empty string if 'desc'
is None
 }
 processed_symbols.append(processed_symbol)
 return processed_symbols

Example usage:
symbols = [{'name': 'Example', 'type': 'Function', 'desc': 'This is an example
function.'}]
processed = process_symbols(symbols)
print(processed)
```
```

Note that this is a simplified representation and actual implementation may vary based on specific requirements and complexity of the `docs/conf.py` module.

3.9 tests/test_hooks.py

The `tests/test_hooks.py` module executes a series of tests to validate the functionality of hook functions. Upon invocation, the module's execution logic proceeds as follows:

- Importing necessary modules:** The module imports the required modules and functions, including the `hook` function, which is a key component of the testing process.
- Defining test cases:** The module defines two test functions: `test_hooks` and `test_default_hooks`. These functions contain the logic for testing the hook functions.
- Test execution:** When the test functions are executed, they invoke the `hook` function with specific input parameters to test its behavior. The `hook` function processes the input data and returns the results, which are then validated by the test functions.
- Assertion and validation:** The test functions assert the correctness of the results returned by the `hook` function, ensuring that it behaves as expected. If the results do not match the expected output, the test fails and an error is reported.
- Test completion:** Once all test cases have been executed, the module reports the test results, indicating whether the tests passed or failed.

In terms of data processing, the `hook` function is the primary component responsible for processing input data. The test functions (`test_hooks` and `test_default_hooks`) provide the input data to the `hook` function and validate its output. The specific data processing logic is as follows:

- Input data:** The test functions provide input data to the `hook` function, which can include various types of data, such as strings, integers, or complex data structures.
- Data processing:** The `hook` function processes the input data according to its implementation logic, which can include operations such as data transformation, filtering, or aggregation.
- Output data:** The `hook` function returns the processed data to the test functions, which then validate the results against the expected output.

Overall, the `tests/test_hooks.py` module provides a robust testing framework for validating the functionality of hook functions, ensuring that they behave correctly and produce the expected results.

| Component | Type | Responsibility |
|-----------|----------|--|
| hook | Function | In the requests system, the Function <code>hook</code> is a callback function that executes at specific points during the request-response cycle. The specific responsibility of the Function <code>hook</code> is to modify or extend the behavior of the request or response object. |

There are two types of hooks: `response` hooks and `request` hooks. `response` hooks are executed after a response is received from the server, while `request` hooks are executed before a request is sent to the server.

The Function `hook` takes a single argument, which is an instance of the `Response` or `Request` object, depending on the type of hook. The hook function can modify the object in place, or return a new object to replace the original one.

The primary use cases for hooks include:

1. Authentication: Hooks can be used to add authentication headers or tokens to requests.
2. Logging: Hooks can be used to log requests and responses for auditing or debugging purposes.
3. Error handling: Hooks can be used to catch and handle exceptions raised during the request-response cycle.
4. Data processing: Hooks can be used to modify or transform the request or response data.

Overall, the Function `hook` provides a flexible way to customize and extend the behavior of the requests system, allowing developers to implement custom logic and workflows. || `test_hooks` | Function | The `test_hooks` function in the requests system is responsible for verifying the correct execution of callback functions registered as hooks.

In the context of the requests library, hooks are callback functions that can be attached to specific events during the request-response cycle, such as 'response' or 'connection:keep-alive'. These hooks enable users to inject custom logic, modify the request or response, or perform additional actions at specific points during the request processing pipeline.

The `test_hooks` function tests the functionality of these hooks by simulating various request scenarios, triggering the registered hooks, and asserting their correct execution. This ensures that the hooks are called at the correct points during the request-response cycle and that they can modify the request or response as expected.

Specifically, `test_hooks` verifies the following:

1. Hook registration: The function ensures that hooks can be successfully registered and that the registration process does not interfere with the normal request processing flow.
2. Hook execution: It verifies that registered hooks are executed at the correct points during the request-response cycle and that they receive the expected arguments.
3. Hook modification: The function checks that hooks can modify the request or response as expected and that these modifications are propagated correctly through the request processing pipeline.

By testing the hooks functionality, `test_hooks` provides assurance that the requests library's event-driven extension mechanism is working correctly and that users can rely on it to customize the request-response cycle according to their needs. || `test_default_hooks` | Function | The `test_default_hooks` function is a unit test designed to verify the functionality of default hooks within the requests system. Its specific responsibility is to test that the default hooks are properly executed during the request-response cycle.

In the context of the requests library, hooks are functions that can be registered to execute at specific points during the request-response cycle, allowing for customization and extension of the library's behavior. Default hooks are those that are registered by default and are executed unless explicitly overridden or removed.

The `test_default_hooks` function is responsible for ensuring that these default hooks are correctly executed, and their effects are properly propagated throughout the system. This includes testing that:

1. Default hooks are registered and executed in the correct order.
2. Default hooks have the expected effects on the request and response objects.
3. Default hooks do not interfere with the normal functioning of the requests library.

By testing the default hooks, this function helps ensure that the requests system behaves as expected and provides a robust and reliable foundation for building applications that rely on it. |

3.10 tests/test_packages.py

The `tests/test_packages.py` module executes a series of tests to verify access to specific attributes within external packages. The execution logic of this module is as follows:

1. Import the required packages, including `urllib3`, `idna`, and `chardet`, which are dependencies to be tested for attribute accessibility.
2. Define the test functions:
 - `test_can_access_urllib3_attribute`: This function tests whether the `urllib3` package is properly imported and if a specific attribute within it is accessible. The test case asserts the existence of the attribute and its correct type or value.
 - `test_can_access_idna_attribute`: Similar to the previous function, this test case verifies the accessibility of an attribute within the `idna` package.
 - `test_can_access_chardet_attribute`: This function checks the accessibility of a specific attribute within the `chardet` package.
3. The test functions utilize assertion statements to validate the accessibility of the specified attributes. If any of these assertions fail, the corresponding test fails, indicating an issue with the package import or attribute access.
4. The module's execution is managed by a testing framework, which discovers and runs the test functions. The framework reports the test results, providing information on whether each test passed or failed.
5. The data processed by this module consists of the packages' attributes, which are checked for accessibility. The test functions do not modify any data; they only verify the presence and accessibility of the specified attributes.

In summary, the `tests/test_packages.py` module executes a series of tests to verify the accessibility of specific attributes within external packages, utilizing assertion statements to validate the results and reporting any failures through a testing framework.

| Component | Type | Responsibility |
|--|----------|---|
| <code>test_can_access_urllib3_attribute</code> | Function | The function <code>test_can_access_urllib3_attribute</code> is a unit test within the requests system, specifically designed to verify that the <code>urllib3</code> attribute is accessible from the <code>requests</code> module. |

This function is responsible for testing that the requests module properly exposes the urllib3 library, ensuring that developers can access its attributes and classes. The test function confirms that the `requests` module's `urllib3` attribute is an instance of the `urllib3` module, validating the integration of urllib3 within the requests library.

Upon execution, this test function asserts the availability and correct configuration of the urllib3 attribute within the requests module, thereby guaranteeing a functional and reliable interface for developers to leverage urllib3's functionality.

In essence, `test_can_access_urllib3_attribute` serves as a quality assurance mechanism, certifying that the requests module adheres to its expected behavior and provides the necessary integration with the urllib3 library. || `test_can_access_idna_attribute` | Function | The function `test_can_access_idna_attribute` is responsible for verifying that the `idna` attribute within the requests system is properly accessible. This function executes a unit test to ensure that the `idna` attribute can be successfully accessed without encountering any errors or exceptions.

In technical terms, this function tests the availability and accessibility of the Internationalized Domain Name Applications (IDNA) attribute, which is a critical component in handling internationalized domain names. By confirming the accessibility of this attribute, the function guarantees that the requests system can properly process and handle IDNA-encoded domain names.

The specific responsibility of this function is to:

1. Initialize a test environment with the necessary prerequisites.
2. Attempt to access the `idna` attribute within the requests system.
3. Verify that the attribute is accessible and does not raise any exceptions.
4. Report the test outcome, indicating success or failure.

By performing this test, the `test_can_access_idna_attribute` function ensures that the requests system is correctly configured and capable of handling IDNA-encoded domain names, which is essential for maintaining a robust and reliable network communication infrastructure. || `test_can_access_chardet_attribute` | Function | The function `test_can_access_chardet_attribute` is a unit test in the requests system, specifically designed to verify that the `chardet` attribute is accessible and properly set on the response object.

This function is responsible for testing the following:

1. The `chardet` attribute is present on the response object.
2. The `chardet` attribute contains the expected character encoding information, as detected by the `chardet` library.

In essence, this test function ensures that the `requests` library correctly integrates with the `chardet` library to detect the character encoding of the response content, and makes this information available through the `chardet` attribute on the response object.

Here's an example of what this function might look like in Python:

```
import requests

def test_can_access_chardet_attribute():
    url = "https://example.com"
```

```
response = requests.get(url)
assert hasattr(response, 'encoding')
assert response.encoding is not None
# Additional assertions may be included to verify the correctness of the
detected encoding
```

This function sends a GET request to the specified URL, retrieves the response object, and asserts that the `encoding` attribute (which is set by the `chardet` library) is present and not `None`. If the assertions pass, it indicates that the `requests` library is correctly accessing and setting the `chardet` attribute on the response object. |

3.11 `src/requests/sessions.py`

The `src/requests/sessions.py` module executes its logic as follows:

1. **Merging Settings:** When a request is made, the `merge_setting` function determines the appropriate setting for that request by taking into account both the explicit setting on the request and the setting in the session. If a setting is a dictionary, the function merges the two dictionaries using the `dict_class` method. This ensures that the request's settings take precedence over the session's settings.
2. **Merging Hooks:** The `merge_hooks` function is responsible for merging both request and session hooks. This is necessary because if the `request_hooks` dictionary contains an empty list as the value for a specific hook (e.g., 'response'), it would break the session hooks entirely. This function ensures that hooks from both the request and session are properly merged.
3. **Session Class:** The `Session` class represents a Requests session, providing cookie persistence, connection-pooling, and configuration. When a request is made through a `Session` object, the object's settings and hooks are merged with those of the request using the `merge_setting` and `merge_hooks` functions, respectively.
4. **SessionRedirectMixin Class:** Although no description is provided, this class likely handles session redirects.
5. **Session Function:** The `session` function returns a `Session` object for context-management. However, this function has been deprecated since version 1.0.0 and should not be used in new code. Instead, the `Session` class should be used directly to create a session.

In terms of data processing, this module handles the following:

- Merging of request and session settings using the `merge_setting` function.
- Merging of request and session hooks using the `merge_hooks` function.
- Creation and management of `Session` objects, which provide cookie persistence, connection-pooling, and configuration for requests.
- Handling of session redirects through the `SessionRedirectMixin` class.

Overall, this module provides essential functionality for managing requests and sessions in a Python application, ensuring that settings and hooks are properly merged and that sessions are handled correctly.

| Component | Type | Responsibility |
|---|----------|---|
| <code>merge_setting</code> | Function | Determines appropriate setting for a given request, taking into account the explicit setting on that request, and the setting in the session. If a setting is a dictionary, they will be merged together using <code>dict_class</code> <code>merge_hooks</code> Function Properly merges both requests and session hooks. |
| This is necessary because when <code>request_hooks == {'response': []}</code> , the merge breaks Session hooks entirely. <code>SessionRedirectMixin</code> Class The <code>SessionRedirectMixin</code> class is responsible for injecting the session cookie into cross-domain redirect responses, enabling cookie persistence across domain changes. | | |
| This mixin specifically handles the case where a redirect response is generated by the requests system and the redirect URL is for a different domain than the original request. By injecting the session cookie into the redirect response, the mixin ensures that the session remains active even when the user is redirected across domains. | | |
| In doing so, <code>SessionRedirectMixin</code> prevents the loss of session data during cross-domain redirects, providing a seamless user experience and ensuring that the session remains intact throughout the request-response cycle. | | |
| This class plays a critical role in maintaining session continuity across domain boundaries, making it an essential component of the requests system. <code>Session</code> Class A Requests session. | | |
| Provides cookie persistence, connection-pooling, and configuration. | | |
| Basic Usage:: | | |
| <pre>import requests s = requests.Session() s.get('https://httpbin.org/get') <Response [200]></pre> | | |
| Or as a context manager:: | | |
| <pre>with requests.Session() as s: s.get('https://httpbin.org/get') <Response [200]></pre> <code>session</code> Function Returns a <code>:class:Session</code> for context-management. | | |
| .. deprecated:: 1.0.0 | | |
| <div>This method has been deprecated since version 1.0.0 and is only kept for backwards compatibility. New code should use <code>:class:`~requests.sessions.Session`</code> to create a session. This may be removed at a future date.</div> | | |
| :rtype: Session | | |

3.12 `src/requests/models.py`

The `src/requests/models.py` module contains the core logic for processing HTTP requests and responses. Here's a breakdown of the execution logic for each class:

1. `RequestEncodingMixin` and `RequestHooksMixin` classes: These two classes are designed to be mixed into the `Request` class to provide additional functionality. The `RequestEncodingMixin` class is responsible for encoding the request data, while the `RequestHooksMixin` class provides hooks for internal usage.
2. `Request` class: The `Request` class is the primary entry point for creating HTTP requests. It takes in several parameters, including `method`, `url`, `headers`, `files`, `data`, `json`, `params`, `auth`, `cookies`, and `hooks`. The execution logic for this class is as follows:
 - Initialize the request object with the provided parameters.
 - If `json` is provided and `files` or `data` is not specified, the `json` parameter is used as the request body.
 - If `data` is a dictionary or a list of tuples, form-encoding takes place.
 - The `prepare()` method is used to prepare the request, which involves creating a `PreparedRequest` object.
3. `PreparedRequest` class: The `PreparedRequest` class represents the fully mutable request object that contains the exact bytes that will be sent to the server. The execution logic for this class is as follows:
 - The `PreparedRequest` object is generated from a `Request` object using the `prepare()` method.
 - The `PreparedRequest` object should not be instantiated manually, as this may produce undesirable effects.
 - The `send()` method of the `Session` class is used to send the `PreparedRequest` object to the server.
4. `Response` class: The `Response` class represents the server's response to an HTTP request. The execution logic for this class is as follows:
 - The `Response` object is generated when the `PreparedRequest` object is sent to the server using the `send()` method.
 - The `Response` object contains the server's response, including the status code, headers, and body.

In terms of data processing, the `Request` class is responsible for encoding the request data, while the `PreparedRequest` class contains the exact bytes that will be sent to the server. The `Response` class contains the server's response, which is processed and returned to the user.

Here's a high-level example of how the data is processed:

```
import requests

# Create a Request object
req = requests.Request('GET', 'https://httpbin.org/get')

# Prepare the Request object
prepped_req = req.prepare()

# Send the PreparedRequest object to the server
s = requests.Session()
resp = s.send(prepped_req)

# Process the Response object
print(resp.status_code)
```

```
print(resp.headers)
print(resp.text)
```

In this example, the `Request` object is created and prepared, and then the `PreparedRequest` object is sent to the server using the `send()` method. The `Response` object is then processed and printed to the console.

| Component | Type | Responsibility |
|-----------------------------------|-------|---|
| <code>RequestEncodingMixin</code> | Class | The <code>RequestEncodingMixin</code> class is responsible for handling content encoding for HTTP requests in the requests system. It provides methods to prepare the request body for transmission by compressing or encoding the data according to the specified encoding type, such as gzip, deflate, or identity. |

Specifically, this class ensures that the request body is properly encoded before being sent over the network, and it sets the `Content-Encoding` header accordingly. This mixin is typically used in conjunction with other request-related classes to ensure that requests are properly formatted and encoded for transmission.

By encapsulating the encoding logic within this mixin, the requests system can easily support various encoding schemes and ensure that requests are properly encoded without affecting the rest of the request processing pipeline. | `RequestHooksMixin` | Class | The `RequestHooksMixin` Class is responsible for injecting hooks into the request lifecycle, enabling the execution of custom callback functions at specific points during the request process.

This mixin class specifically provides methods for registering hooks that can be triggered during the following events:

- Response generation: The `response` hook is called after a response has been generated, but before it is sent back to the client.
- Request exception: The `request` hook is called when an exception occurs during the request process.

By providing these hook points, `RequestHooksMixin` allows developers to extend and customize the request handling behavior of the system, enabling advanced use cases such as logging, authentication, and rate limiting.

The mixin is designed to be composed with other request-related classes, providing a modular and reusable way to add hook functionality to the request lifecycle. By leveraging this mixin, developers can create custom request handlers that are highly adaptable and extensible.

Example of how `RequestHooksMixin` can be used:

```
class MyRequestHandler(RequestHooksMixin):
    def __init__(self):
        self.hooks = {}

    def register_hook(self, event, callback):
        if event not in self.hooks:
            self.hooks[event] = []
        self.hooks[event].append(callback)
```

```

def dispatch_hook(self, event, *args, **kwargs):
    if event in self.hooks:
        for callback in self.hooks[event]:
            callback(*args, **kwargs)

def send_request(self, request):
    # Send the request...
    response = requests.get('https://example.com')

    # Dispatch the response hook
    self.dispatch_hook('response', response)

    return response

# Create a request handler
handler = MyRequestHandler()

# Register a response hook
def on_response(response):
    print(f'Received response: {response.status_code}')

handler.register_hook('response', on_response)

# Send a request
handler.send_request(requests.Request('GET', 'https://example.com'))

```

In this example, `RequestHooksMixin` is used to create a custom request handler that supports hooks. The `register_hook` method is used to register a response hook, which is then dispatched when a response is received. This demonstrates the mixin's ability to extend and customize the request handling behavior of the system. | `Request` | Class | A user-created `:class:Request` `<Request>` object.

Used to prepare a `:class:PreparedRequest` `<PreparedRequest>`, which is sent to the server.

:param method: HTTP method to use. :param url: URL to send. :param headers: dictionary of headers to send. :param files: dictionary of {filename: fileobject} files to multipart upload. :param data: the body to attach to the request. If a dictionary or list of tuples `[(key, value)]` is provided, form-encoding will take place. :param json: json for the body to attach to the request (if files or data is not specified). :param params: URL parameters to append to the URL. If a dictionary or list of tuples `[(key, value)]` is provided, form-encoding will take place. :param auth: Auth handler or (user, pass) tuple. :param cookies: dictionary or CookieJar of cookies to attach to this request. :param hooks: dictionary of callback hooks, for internal usage.

Usage::

```

import requests req = requests.Request('GET', 'https://httpbin.org/get') req.prepare()
<PreparedRequest [GET]> | PreparedRequest | Class | The fully mutable
:class:PreparedRequest <PreparedRequest> object, containing the exact bytes that
will be sent to the server.

```

Instances are generated from a `:class:Request` `<Request>` object, and should not be instantiated manually; doing so may produce undesirable effects.

Usage::

```
import requests req = requests.Request('GET', 'https://httpbin.org/get') r = req.prepare()  
r <PreparedRequest [GET]>
```

```
s = requests.Session() s.send(r) <Response [200]> | | Response | Class | The  
:class:Response <Response> object, which contains a server's response to an HTTP  
request. |
```